

Formal Creativity for MAS-LLMs: A Lean-Checked Bridge from Information Objectives to Benchmark Claims

Abstract

We introduce Creative Update Efficiency (CUE), a receiver-grounded creativity metric defined as Brier-score delta per output bit: an output is credited only for the decision-quality improvement it induces in a receiver agent, normalized by its length in bits. CUE is proved to be a lower bound on normalized Shannon Value of Information (`CUE_voi_equivalence`) and bounded by the Shannon maximum $\log |\mathcal{Z}|/|y|_{\text{bits}}$ (`shannon_upper_bound_normalization`), where the maximum-entropy bound is itself derived from the Gibbs inequality rather than assumed. The reported **Benchmark Score** is defined by

$$R_{\text{creativity}}(q, y, s) = 1[\text{CUE}(y, A, T) > 0] \cdot 1[R_D(y, \mathcal{P}) > \delta_D] \cdot \left(\text{CUE}(y, A, T) \cdot (1 + \alpha R_B^{\rightarrow A}(y, A)) + \lambda_G G_k(q, y) \right)$$

(**Rcreativity**): both $1[\text{CUE} > 0]$ and $1[R_D > \delta_D]$ are multiplicative safety floors on the *entire* score — including $\lambda_G G_k$ — so neither CUE failure nor D-gate failure leaves an additive escape hatch. There is no compression-novelty term R_C . Boundedness, monotonicity in CUE, and total-score gate closure are machine-checked. Trajectory diagnostics (Step-CUE, Diverge-Converge) and a multi-agent bridge ($G_k \Rightarrow$ joint CUE improvement) accompany the score. A Dual-Process Separation Principle is proved as a structural theorem: the training signal $R_{\text{train}} = R_B + \tilde{\lambda}_D \tilde{R}_D$ depends only on live resampled quantities, so its gradient is independent of every frozen benchmark object, and a Structural Goodhart Collapse theorem shows that violating any protocol condition admits a rank reversal that refutes the proxy-faithfulness guarantee. The Lean 4 development certifies these claims; remaining assumptions are centralized in a Known Gaps Registry. The document separates frozen benchmark objectives (Part III) from the 2-CGRPO training signal (Part V); machine-checked Goodhart-resistance conditions and a monitorable proxy faithfulness invariant (Part VI) govern when training results may be compared against the benchmark.

1 The Problem with Scalar Creativity Evaluation

Current MAS-LLM evaluation often treats creativity as a scalar quality score: a human or LLM judge rates an output against a rubric. This is insufficient for three reasons.

First, it is reductive. A scalar score collapses a multi-dimensional property into a single number and hides the distinction between novelty, usefulness, diversity, surprise, downstream impact, and interaction structure.

Second, it is evaluator-dependent. LLM judges can over-rate LLM-generated ideas relative to expert human evaluation, producing a novelty mirage: outputs appear creative to the evaluator because they are fluent or stylistically unusual, not because they introduce new information.

Third, scalar evaluation misses the point of multi-agent systems. Creativity in a collective system is not merely the sum of individual creative outputs. The central object is what emerges from interaction that no individual agent could produce alone.

Claim 1 (Core thesis). *Creativity in MAS-LLMs should be defined as a property of the system’s information dynamics, not as a property of individual outputs.*

Framework Preview

We define creativity as utility-gated informational contribution. The feasibility condition is

$$\text{Feas}(q, y) \iff U(q, y) \geq \tau.$$

Within the feasible set, B measures downstream expansiveness and D measures probe-relative structural influence on a fixed hold-out probe set $\mathcal{P}$ (defined by the protocol in Appendix A). The sole reported score is the **Benchmark Score**

$$R_{\text{creativity}}(q, y, s) := 1[\text{CUE}(y, A, T) > 0] \cdot 1[R_D(y, \mathcal{P}) > \delta_D] \cdot \left(\text{CUE}(y, A, T) \cdot (1 + \alpha R_B^{\rightarrow A}(y, A)) + \lambda_G G_k(q, y) \right), \quad (1)$$

formalized as **Rcreativity** (Section 4): **cueGate** and **dGate** multiply the whole bracket $\text{CUE} \cdot (1 + \alpha R_B^{\rightarrow A}) + \lambda_G G_k$, enforcing a strict safety floor (either gate failure zeroes the score, including interaction gain). There is no compression-novelty component R_C . An optional additive B–D diagnostic $R_B + \lambda_D R_D$ (**benchmarkReward**) supports rank-preservation correspondence and is *not* $R_{\text{creativity}}$.

This is not the training objective. R_D requires a frozen probe set $\mathcal{P}$ and must never receive gradient updates. The training objective is $R_{\text{train}} = R_B + \tilde{\lambda}_D \tilde{R}_D$, defined in Part V.

Contributions Hierarchy

Contributions, in order of priority. **1. Primary — CUE as VOI-grounded multiplicative base (new).** Creative Update Efficiency, receiver decision-quality gain per output bit, proved to lower-bound normalized Shannon Value of Information (**CUE_voi_equivalence**) and to respect the Shannon ceiling (**shannon_upper_bound_normalization**); the reported Benchmark Score is (1) (**Rcreativity**). **2. Secondary — Dual-Process Separation Principle.** Goodhart resistance as an *architectural theorem* (**dual_process_separation_principle**): the training gradient is structurally independent of frozen benchmark objects, paired with the Structural Goodhart Collapse theorem (**structural_goodhart_collapse**) showing each protocol violation admits a refuting rank reversal. **3. Tertiary — Trajectory diagnostics.** The Step-CUE curve is monotone under the per-step gate (**CUE_trajectory_monotone_nondecreasing**), its saturation rate γ is positive exactly under continuous discovery (**step_CUE_gamma_positivity_characterization**), and the Diverge-Converge score is achievable and non-trivial (**DC_score_existence**, **monotone_increasing_not_DC**). **4. Supporting — B/D information-theoretic scaffolding.** Entropy monotonicity, mixture anti-collapse (now axiom-free), utility gating, probe-relative deformation, baseline contrast, estimator rank preservation, and the multi-agent interaction-gain bridge.

Relationship to ComplLLM

ComplLLM (Guo et al., 2026) post-trains a decision-assistant LLM using complementary information as reward: an output scores only if it complements existing agent decisions, validated in verifiable decision-support domains. CUE inherits from ComplLLM the receiver-outcome focus and the

use of decision quality as a training signal. It generalizes ComplLLM in three ways: (a) it applies to open-ended creative tasks without binary payoff decompositions, because the Brier-delta is defined against any finite decision-relevant state space; (b) it normalizes by output length in bits and connects the normalized quantity to a Shannon upper bound (`shannon_upper_bound_normalization`); and (c) it is embedded in a dual-process Goodhart-resistance protocol that separates frozen benchmark objects from live training proxies (Part IV).

English Roadmap of the Lean Argument

The Lean development supports this thesis by proving that the proposed information-dynamic objectives behave exactly as the paper claims. The proof strategy is simple in spirit. First, define a reward B that gives credit to outputs only when they pass a utility gate, and then ranks feasible outputs by the entropy of the downstream states they induce. Lean proves that once the gate is open, higher downstream entropy strictly means higher B reward. It also proves that a lower-entropy feasible output cannot be optimal when a higher-entropy feasible output exists. If the high-entropy output receives sufficient probability mass, mixing over feasible outputs cannot do worse than collapsing to a strictly dominated lower-entropy output. In ordinary terms: B formally rules out the kind of collapse where a system keeps choosing a safe, narrow, predictable answer despite the availability of a more generative feasible alternative.

Second, define a reward C that gives credit to outputs only when they pass the same utility gate, and then ranks them by how costly they are to encode relative to an existing corpus. Lean proves that C gives zero reward to infeasible outputs no matter how novel they look, that exact copies are bounded by the compressor’s copy-minimality axiom, and that padding with zero-information content cannot improve the novelty score. In ordinary terms: C separates genuine novelty from three common failure modes: nonsense, plagiarism, and length padding. This is exploratory creativity in Boden’s sense: novelty *within* a fixed conceptual space.

Third, for empirical implementation, define surface novelty N as KL divergence between an output’s n -gram PMF and the corpus PMF. N is not a fourth creativity dimension; it is the tractable surface approximation to C . Lean proves its gate, nonnegativity, and cross-entropy link in the computational approximations appendix.

Fourth, define D as probe-relative structural influence: corpus deformation gain on a fixed finite probe set $\mathcal{P}$. Under `ProbeCompressor`, Lean proves that D is utility-gated, order-preserving, nonnegative, and zero for exact corpus copies, for any fixed $\mathcal{P}$. The benchmark operationalizes $\mathcal{P}$ by the hold-out protocol in Appendix A. High D correlates with transformational creativity when $\mathcal{P}$ is domain-representative; the formal theorems do not identify D with Boden–Wiggins transformation outright. High C measures departure from the current corpus; high D measures how much s changes compression of the frozen probes.

Fifth, Lean proves that these objectives add something real beyond obvious baselines. A utility-only reward cannot distinguish two feasible outputs with the same utility but different downstream entropy; B can. A novelty-only reward can assign positive score to an infeasible string; C cannot. D adds a separate probe-relative invariant: it measures whether incorporating an output changes compression of a frozen hold-out probe set, while Lean proves the score is utility-gated, order-preserving, nonnegative, and zero for exact corpus copies. These contrast theorems are important because they show that B , C , and D are not just renamings of quality or novelty rubrics; the surface- N proxy inherits the same gate and nonnegativity discipline in the appendix. They enforce different invariants.

Sixth, Lean bridges the abstract objectives to measurement and training. If a benchmark estimator has uniformly bounded error, then entropy and novelty rankings are preserved whenever

the true gap is larger than twice the error bound. This gives the benchmark an explicit resolution condition: when measured differences exceed 2ϵ , the ranking is not an artifact of estimator noise. Lean also proves that a simplified DPO update constructed from a B-valid preference pair increases the winner’s log-odds over the loser, so the training signal moves in the direction certified by the objective. The additive `benchmarkReward` diagnostic combines B and D for group-relative RL correspondence; the reported score remains (1).

Finally, Lean extends the argument to multi-agent settings. It defines an interaction gain: the difference between joint downstream entropy and the best individual downstream entropy. If this gain is positive, Lean proves that neither individual agent alone achieves the joint entropy. The gain is zero exactly in the degenerate case where the joint entropy equals the best individual entropy (`gain_zero_iff_joint_eq_max`); notably, independent product kernels with positive marginals have strictly positive gain by entropy additivity, which is why the framework pairs the gain with the faithfulness assumption and the receiver-level CUE bridge (`Gk_implies_CUE_improvement`) rather than treating positive gain alone as synergy. If the gain exceeds twice the benchmark error, Lean proves that the estimator detects this irreducibility. The k -agent extension and O-information sign bridge are the general versions; the two-agent case is the readable base case.

Taken together, the Lean proofs establish the paper’s central logical chain: utility-gated entropy prevents collapse; utility-gated compression novelty prevents nonsense, copying, and padding; bounded estimators preserve the theoretical rankings; DPO-style training moves in the certified direction; and faithful non-product multi-agent kernels provide a formal proxy for emergent coordination.

Part I

Receiver-Grounded Creativity Foundation

This part is the definitional anchor of the framework. Creativity is grounded in the *receiver*: an output is creative only insofar as it improves a receiver agent’s downstream decisions, efficiently per bit. Everything else in the document — the B/C/D components, the trajectory diagnostics, the multi-agent gain, the training signal — is subordinate to this gate.

2 Creative Update Efficiency (CUE)

Definition 1 (CUE model). *CUEModel*. A receiver-grounded CUE model over a finite decision-relevant state space $\mathcal{Z}$ carries the receiver’s posterior belief after the output, the Brier-score improvement $\Delta_{\text{Brier}}(y) \geq 0$ from prior to posterior, the Shannon value of information $\text{VOI}(y)$ of the output for the receiver’s decision, and the output length $|y|_{\text{bits}} > 0$. Two structural facts are carried as certified fields: the proper-scoring-rule bound $\Delta_{\text{Brier}} \leq \text{VOI}$, and the residual-entropy budget $\text{VOI} \leq H(\text{posterior})$.

Definition 2 (Creative Update Efficiency). *cue*. For a CUE model m ,

$$\text{CUE}(y, A, T) := \frac{\Delta_{\text{Brier}}(y)}{|y|_{\text{bits}}},$$

the receiver decision-quality gain per output bit. Nonnegativity is `cue_nonneg`.

Theorem 1 (PROOF-01: CUE lower-bounds normalized VOI). *CUE_voi_equivalence*. For any CUE model,

$$\text{CUE}(y, A, T) \leq \frac{\text{VOI}(y, A, T)}{|y|_{\text{bits}}},$$

with equality when the Brier improvement saturates its proper-scoring-rule bound — the calibrated binary case (*CUE_voi_equality*).

Proof. Divide the proper-scoring-rule bound $\Delta_{\text{Brier}} \leq \text{VOI}$ by the positive bit length; Lean closes with `div_le_div_iff_of_pos_right`. Equality substitutes $\Delta_{\text{Brier}} = \text{VOI}$. $\square$

Theorem 2 (Maximum entropy on a finite alphabet). *maxEntropy_finite_alphabet*. For any PMF p over a finite nonempty alphabet $\mathcal{Z}$,

$$H(p) \leq \log |\mathcal{Z}|.$$

Proof. No new axiom is introduced: the KL divergence of p against the uniform distribution evaluates to $\log |\mathcal{Z}| - H(p)$ (`klDivergence_uniform_eq`, using the finite mass identity `sum_toReal_eq_one`), and the existing Gibbs inequality `klDivergence_nonneg` makes it nonnegative. $\square$

Theorem 3 (PROOF-02: Shannon upper bound normalization). *shannon_upper_bound_normalization*. For any CUE model over $\mathcal{Z}$,

$$\text{CUE}(y, A, T) \leq \frac{\log |\mathcal{Z}|}{|y|_{\text{bits}}}.$$

Proof. Chain $\Delta_{\text{Brier}} \leq \text{VOI} \leq H(\text{posterior}) \leq \log |\mathcal{Z}|$, the last step by `maxEntropy_finite_alphabet`, and divide by the positive bit length. $\square$

Theorem 4 (CUE primary gate). *CUE_primary_gate*. Zero or negative CUE disqualifies all downstream metrics: if $\text{CUE}(y, A, T) \not\geq 0$, the gated creativity score of y is zero regardless of its B , C , or D values (*cueGatedScore*).

CUE gates everything. The B/C/D components of Part II are *components of the CUE-gated score*, not co-equal creativity dimensions. An output that fails the CUE gate receives no creativity credit no matter how expansive, novel, or structurally influential it is.

3 Receiver Expansion vs. Clarification

The bidirectional receiver label $R_B^{\rightarrow A}$ records whether an output *expands* the receiver’s downstream generative space (high receiver entropy) or *clarifies* it (low receiver entropy).

Theorem 5 (PROOF-06: expansion/clarification separation). *receiver_expansion_clarification_separation*. The label $R_B^{\rightarrow A}$ is not a monotone function of CUE: there exist outputs y_1, y_2 with equal CUE but opposite labels — one expanding ($R_B^{\rightarrow A}(y_1) > \delta_E$) and one clarifying ($R_B^{\rightarrow A}(y_2) < \delta_E$).

Proof. Explicit finite construction mirroring `BC_tension_example`: two labeled outputs (`LabeledOutput`) share the same underlying CUE model on `Fin(2)` (hence genuinely equal CUE), while their receiver expansion magnitudes are 1 (maximal, uniform-generation scale) and 0 (point-mass commitment), with threshold $\delta_E = 1/2$ strictly between. $\square$

4 The Benchmark Score: CUE as Multiplier

Canonical definition. The reported Benchmark Score is exactly (1):

$$R_{\text{creativity}}(q, y, s) = \mathbb{I}[\text{CUE}(y, A, T) > 0] \cdot \mathbb{I}[R_D(y, \mathcal{P}) > \delta_D] \cdot \left(\text{CUE}(y, A, T) \cdot (1 + \alpha R_B^{\rightarrow A}(y, A)) + \lambda_G G_k(q, y) \right).$$

Lean name: `Rcreativity` (abbrev. `benchmarkScore`). Gates are `cueGate` and `dGate`. There is no R_C term. Placing $\lambda_G G_k$ *inside* both indicators closes the additive escape hatch that an ungated interaction-gain term would create: any CUE or R_D violation immediately nullifies the total reward (multiplicative safety floor). Proof obligations of this regime are discharged as follows.

Theorem 6 (CUE-gate closure — total zero). *`Rcreativity_cue_gate_closed`. If $\text{CUE} \leq 0$, then $R_{\text{creativity}} = 0$ (including the $\lambda_G G_k$ term).*

Theorem 7 (D-gate closure — total zero). *`Rcreativity_d_gate_closed`. If $R_D \leq \delta_D$, then $R_{\text{creativity}} = 0$ (including the $\lambda_G G_k$ term). The magnitude of R_D never enters the score (`D2_dominance` is diagnostic-only here).*

Theorem 8 (Nonnegativity). *`Rcreativity_nonneg`. Under $\alpha > 0$, $r_b \in [0, 1]$, $0 \leq \lambda_G$, and $0 \leq G_k$, one has $0 \leq R_{\text{creativity}}$.*

Theorem 9 (CUE-as-multiplier ceiling). *`Rcreativity_bounded`. $R_{\text{creativity}} \leq (\log |Z|/|y|_{\text{bits}}) \cdot (1 + \alpha) + \lambda_G G_k$, via `shannon_upper_bound_normalization` and `receiver_expansion_bounded`.*

Theorem 10 (Monotonicity in CUE). *`Rcreativity_monotone_in_CUE`. With both gates open and fixed $(\alpha, r_b, \lambda_G, G_k)$, strictly larger CUE yields a strictly larger score: CUE’s magnitude is load-bearing.*

Theorem 11 (Positivity matches CUE). *`Rcreativity_pos_iff_CUE_pos`. With the D-gate open and $\lambda_G G_k = 0$, one has $0 < R_{\text{creativity}}$ iff $0 < \text{CUE}$.*

Theorem 12 (Unit reconciliation). *`unit_conversion_sign_and_order_invariant`. Positive rescaling preserves sign and order. CUE is bits-normalized while $R_B^{\rightarrow A}$ is nats-based; the residual scale is absorbed by the free-fit hyperparameter α (KG-6), explicitly weaker than a proved λ -normalization.*

Proof flow (central claims).

1. *CUE is a valid base:* `PROOF-01/02` give VOI lower-bound and Shannon ceiling; `cueGate` is a safety floor on the whole score.
2. *R_D is a filter, not a summand:* `dGate` likewise zeroes the entire score; magnitude is discarded (`D2_dominance` diagnostic-only). The former R_C -coupled D3 bound is replaced by `D3_rewardD_bound_standalone`.
3. *Expansion bonus is controlled:* `receiver_expansion_bounded` gives $r_b \in [0, 1]$, so the factor $1 + \alpha r_b$ lies in $[1, 1 + \alpha]$. Theorem 5’s separate-label result (`receiver_expansion_clarification_separation`) remains valid as a diagnostic statement; folding $R_B^{\rightarrow A}$ into (1) is a deliberate, documented exception for the reported score.
4. *G_k has no ungated path:* $\lambda_G G_k$ sits inside both indicators, so Theorems 6–7 prove total-score zero on either gate failure (strengthening the earlier “reduces to $\lambda_G G_k$ ” statements). `Gk_implies_CUE_improvement` / `Gk_gated_by_CUE_shifted` remain compatible with this floor.

5. Former R_C dependencies are bridged: `lambda_D_normalization_standalone` and `paretoFrontier_nonempty_BD` replace λ_C - and three-way-C-coupled commensurability/Pareto claims for any residual B–D diagnostic.

Optional shifted calibration. The scalar $\text{CUE}_f := \text{CUE} - f$ (`CUE_shifted`, KG-5) and `Rcreativity_shifted` are retained only as calibration variants; they are *not* the canonical definition (1).

Theorem 13 (Shifted Shannon ceiling). *`CUE_shifted_bound`. For every CUE model m and threshold f , $\text{CUE}_f \leq \log |Z|/|y|_{\text{bits}} - f$.*

Theorem 14 (Universal expansion bound). *`receiver_expansion_bounded`. With $R_B^{\rightarrow A} = H(\text{Gen}(A \mid y, T))/\log |W|$ (`receiverExpansionNorm`), one has $0 \leq R_B^{\rightarrow A} \leq 1$.*

Theorem 15 (G_k gated by shifted CUE). *`Gk_gated_by_CUE_shifted`. Under `Gk_implies_CUE_improvement`, if $G_k > 0$ and $f < \text{CUE}$, then $G_k \cdot \mathbb{I}[\text{CUE}_f > 0] > 0$.*

5 Formal Correspondence Table

Proposal construct	Lean theorem	Status
<code>CUE(y, A, T)</code>	<code>cue</code> , <code>CUE_voi_equivalence</code>	Machine-checked (PROOF-01)
Shannon normalization	<code>shannon_upper_bound_normalization</code>	Machine-checked (PROOF-02)
Benchmark Score	<code>Rcreativity</code>	Canonical def. (1)
$R_{\text{creativity}}$		
<code>CUE / D gates (total zero)</code>	<code>cueGate</code> , <code>dGate</code> , <code>Rcreativity_cue_gate_closed</code> , <code>Rcreativity_d_gate_closed</code>	Score = 0 if either fails
Boundedness / monotone	<code>Rcreativity_bounded</code> , <code>Rcreativity_monotone_in_CUE</code>	Machine-checked
Positivity	<code>Rcreativity_pos_iff_CUE_pos</code>	Machine-checked
Unit reconciliation	<code>unit_conversion_sign_and_order_invariant</code>	Machine-checked
$R_B^{\rightarrow A}$ bound	<code>receiver_expansion_bounded</code>	Machine-checked
G_k decision bridge	<code>Gk_implies_CUE_improvement</code> , <code>Gk_gated_by_CUE_shifted</code>	Machine-checked (PROOF-07)
D3 standalone	<code>D3_rewardD_bound_standalone</code>	Bridge (R_C -free)
λ_D standalone	<code>lambda_D_normalization_standalone</code>	Bridge (B–D diagnostic)
B–D Pareto	<code>paretoFrontier_nonempty_BD</code>	Bridge (B–D diagnostic)
DPS / Goodhart	<code>dual_process_separation_principle</code> , <code>structural_goodhart_collapse</code>	Machine-checked

Part II

Creativity Definition (Measurement)

This part is organized in three tiers subordinate to the CUE foundation of Part I: **Tier A (multiplicative base)** is gated CUE; **Tier B (structural filters)** comprises the R_D probe-relative gate and the $R_B^{\rightarrow A}$ expansion bonus; **Tier C (interaction certification)** is the multi-agent gain G_k . The sole reported measurement is the Benchmark Score (1) (`Rcreativity`, Section 4). It is *not* the quantity the trainer optimizes: the training signal $R_{\text{train}} = R_B + \tilde{\lambda}_D \tilde{R}_D$ (Part V) replaces frozen R_D by the live proxy. The additive intermediate $R_B + \lambda_D R_D$ (`benchmarkReward`) serves B/D rank-preservation correspondence only and is never identified with $R_{\text{creativity}}$. Compression novelty R_C is not a score component; `AbstractCompressor` remains solely as infrastructure for `ProbeCompressor/R_D`.

6 Formal Framework

6.1 The B Reward

The Lean structure `BModel` contains a kernel

$$\text{kernel} : Q \rightarrow Y \rightarrow \text{PMF}(Z),$$

a utility function $U : Q \rightarrow Y \rightarrow \mathbb{R}$, and a threshold $\tau \in \mathbb{R}$. The project defines

$$\text{Feas}_m(q, y) \iff \tau \leq U(q, y)$$

and

$$R_B(q, y) := \begin{cases} H_q(y), & \text{Feas}_m(q, y), \\ 0, & \text{otherwise.} \end{cases}$$

6.2 Compressor infrastructure (not a score component)

R_C is *not* part of $R_{\text{creativity}}$. The Lean class `AbstractCompressor` is retained solely so that `ProbeCompressor` (and hence R_D) can use code-length / novelty primitives:

1. a code length function $L : \text{List}(\alpha) \rightarrow \mathbb{R}$;
2. nonnegativity: $0 \leq L(s)$ for all strings s ;
3. copy minimality:

$$L(\mathcal{H} ++ s) - L(\mathcal{H}) \leq L(s).$$

The novelty score is

$$N(s \mid \mathcal{H}) = L(\mathcal{H} ++ s) - L(\mathcal{H}),$$

and the gated reward is

$$R_C(s, \mathcal{H}, u, \tau) := \begin{cases} N(s \mid \mathcal{H}), & \tau \leq u, \\ 0, & \text{otherwise.} \end{cases}$$

This R_C object is *not* used in (1); it is shown only because $N(\cdot \mid \mathcal{H})$ underpins R_D . For padding results, `NormalizedCompressor` adds:

$$L(\mathcal{H} ++ (s ++ p)) \leq L(\mathcal{H} ++ s) + L(p).$$

6.3 The D Reward

The Lean class `ProbeCompressor` extends the compressor setting with a fixed finite probe set $\mathcal{P}$. The deformation score measures how much incorporating s into the corpus reduces compression cost on those probes:

$$D(s, \mathcal{H}, \mathcal{P}) := \sum_{s' \in \mathcal{P}} (N(s' \mid \mathcal{H}) - N(s' \mid \mathcal{H} \cup \{s\})).$$

The gated reward is

$$R_D(s, \mathcal{H}, \mathcal{P}, u, \tau) := \begin{cases} D(s, \mathcal{H}, \mathcal{P}), & \tau \leq u, \\ 0, & \text{otherwise.} \end{cases}$$

D therefore measures probe-relative structural influence: it is not novelty relative to the current corpus, but the extent to which adding the output changes compression of a frozen hold-out probe set.

6.4 Combined Training Objective

In a DPO-style training regime, preference pairs are built inside the feasible set. A B-pair prefers the feasible output with higher downstream entropy; a D-pair prefers the feasible output with higher probe-relative deformation gain. The conceptual combined loss is

$$\mathcal{L}(\theta) = \mathcal{L}_{B\text{-DPO}}(\theta) + \lambda_D \mathcal{L}_{D\text{-DPO}}(\theta),$$

with no R_C / C-DPO channel. B governs downstream impact and D governs structural influence on a fixed probe set. Reported evaluation uses $R_{\text{creativity}}$ (1), not this training loss.

6.5 Commensurability: B–D diagnostic normalization

The optional additive diagnostic $R_B + \lambda_D R_D$ is justified because $H_q(y)$ and $D(s, \mathcal{H}, \mathcal{P})$ are Shannon quantities in nats; λ_D is a dimensionless scale. The reported Benchmark Score is multiplicative (1) and instead uses the unit-reconciliation lemma of Section 4. For the diagnostic:

Theorem 16 (B ceiling). *`rewardB_le_maxEntropy`. For every prompt and output, $R_B(q, y) \leq \log |\mathcal{Z}|$: gated-off outputs score $0 \leq \log |\mathcal{Z}|$ and feasible outputs score their downstream entropy, bounded by `maxEntropy_finite_alphabet` (Part I).*

Theorem 17 (λ_D normalization). *`lambda_D_normalization_standalone`. With $\lambda_D = \log |\mathcal{Z}| / (|P| \cdot \log |\Sigma| \cdot \mathbb{E}[|s'|])$, where $|P|$ is the probe set size and $\mathbb{E}[|s'|]$ is the mean probe string length,*

$$\lambda_D \cdot (|P| \cdot \log |\Sigma| \cdot \mathbb{E}[|s'|]) = \log |\mathcal{Z}|,$$

so the maximal λ_D -scaled D contribution coincides exactly with the proved B ceiling.

R_B and R_D are two projections of the same generative event: the agent’s output simultaneously induces a downstream state distribution (measured by B) and deforms the compression of domain-representative probes (measured by D); normalization ensures neither projection dominates by default in the diagnostic.

6.6 Summary Criterion

In this framework, a MAS-LLM behavior is creative when it scores positively on (1): both CUE and R_D gates open (else the score is identically zero), with CUE magnitude, expansion bonus, and (when applicable) interaction gain G_k contributing inside that floor. High D on a domain-representative $\mathcal{P}$ is an empirical correlate of transformational creativity rather than transformation by definition.

7 Lean-Checked Objective Guarantees

This section records the exact mathematical assumptions under which the Lean project proves each objective property. The proof checker verifies the statements listed by their Lean names.

7.1 Entropy Layer

Theorem 18 (Shannon entropy nonnegativity). *`shannonEntropy_nonneg`. For any finite type α and any $p : \text{PMF}(\alpha)$,*

$$0 \leq - \sum_{x:\alpha} p(x) \log p(x).$$

Proof. For each x , $p(x)$ is a probability mass, so $0 \leq p(x) \leq 1$. Hence $\log p(x) \leq 0$ and therefore $p(x) \log p(x) \leq 0$. Summing over the finite type gives

$$\sum_x p(x) \log p(x) \leq 0.$$

Multiplying by -1 yields

$$0 \leq -\sum_x p(x) \log p(x).$$

In Lean this is proved by applying `Real.mul_log_nonpos` pointwise and then `Finset.sum_nonpos`. $\square$

Theorem 19 (Point mass entropy). *shannonEntropy_point_mass.* For any finite type α and $x_0 : \alpha$,

$$H(\delta_{x_0}) = 0.$$

Proof. For the point mass δ_{x_0} , the probability is 1 at x_0 and 0 at every other point. Thus every entropy summand is either $1 \log 1$ or $0 \log 0$, both of which evaluate to 0 in the real-valued definition used by the project. Therefore the finite sum is zero. Lean proves this by splitting on whether $x = x_0$ and simplifying `PMF.pure_apply`. $\square$

Theorem 20 (Entropy order unfolding). *shannonEntropy_lt_iff.* For finite α and $p, q : \text{PMF}(\alpha)$,

$$H(p) < H(q) \iff -\sum_x p(x) \log p(x) < -\sum_x q(x) \log q(x).$$

Proof. This is definitional: `shannonEntropy` is exactly the finite negative sum on both sides. Lean proves the equivalence by simplifying with the definition of `shannonEntropy`. $\square$

7.2 Lean Helper Lemmas for B

The B model file also proves four helper lemmas used by later theorems.

Theorem 21 (Reward on feasible outputs). *rewardB_of_feasible.* If $\text{Feas}_m(q, y)$, then

$$R_B(q, y) = H_q(y).$$

Proof. The definition of R_B is an if-then-else on feasibility. Under the assumption $\text{Feas}_m(q, y)$, the true branch is selected. Lean proves this by simplifying `rewardB`. $\square$

Theorem 22 (Reward on infeasible outputs). *rewardB_of_infeasible.* If $\neg \text{Feas}_m(q, y)$, then

$$R_B(q, y) = 0.$$

Proof. Unfold the same if-then-else defining R_B . Under infeasibility, the false branch is selected, which is 0. Lean proves this by simplification. $\square$

Theorem 23 (B reward nonnegativity). *rewardB_nonneg.* For every q, y ,

$$0 \leq R_B(q, y).$$

Proof. If y is feasible, then $R_B(q, y) = H_q(y)$, which is nonnegative by `shannonEntropy_nonneg`. If y is infeasible, then $R_B(q, y) = 0$. Lean proves this by case-splitting on feasibility and simplifying. $\square$

Theorem 24 (Feasible-set membership). *mem_feasibleSet_iff. For finite Y ,*

$$y \in \text{feasibleSet}(q) \iff \text{Feas}_m(q, y).$$

Proof. The feasible set is defined as the finite universe filtered by the feasibility predicate. Membership in this filtered universe is therefore equivalent to feasibility itself. Lean proves this by simplifying `feasibleSet`. $\square$

7.3 B: Anti-Collapse and Entropic Ordering

Theorem 25 (B1: dominance). *B1_dominance. Assume Z is finite, $m : \text{BModel}(Q, Y, Z)$, $q : Q$, and $y_1, y_2 : Y$. If*

$$\text{Feas}_m(q, y_1), \quad \text{Feas}_m(q, y_2), \quad H_q(y_2) < H_q(y_1),$$

then

$$R_B(q, y_2) < R_B(q, y_1).$$

Proof. Because both outputs are feasible, the utility gate is open for both:

$$R_B(q, y_i) = H_q(y_i) \quad (i = 1, 2).$$

Substituting these equalities into the desired conclusion reduces it exactly to the hypothesis $H_q(y_2) < H_q(y_1)$. Lean proves this by simplifying the definition of `rewardB` under the feasibility hypotheses. $\square$

Consequence: inside the feasible set, B rewards exactly the downstream entropy ordering. The result is order-theoretic: once the gate is open, the reward unfolds to entropy.

Theorem 26 (B2: no entropy-dominated optimum). *B2_no_collapse. Assume Y and Z are finite. Let $m : \text{BModel}(Q, Y, Z)$, $q : Q$, and $y, y^* : Y$. If*

$$\text{Feas}_m(q, y), \quad \text{Feas}_m(q, y^*), \quad H_q(y) < H_q(y^*),$$

and the feasible set $\{y' : \text{Feas}_m(q, y')\}$ is nonempty, then

$$R_B(q, y) < \sup_{y' \in \text{Feas}_m(q, \cdot)} R_B(q, y').$$

Proof. By B1, the hypotheses imply

$$R_B(q, y) < R_B(q, y^*).$$

Since y^* is feasible, it belongs to the feasible finite set. A finite supremum is strictly above a value whenever some member of the set is strictly above that value. Applying this finite-supremum fact to y^* gives

$$R_B(q, y) < \sup_{y' \in \text{Feas}_m(q, \cdot)} R_B(q, y').$$

Lean uses `Finset.lt_sup' _iff` with witness y^* . $\square$

Consequence: a deterministic policy concentrated on a strictly entropy-dominated feasible output cannot be optimal for B.

Mixture rewards (B3)

Proof status (PROOF-09, resolved). The distributional Jensen extension `B3b_jensen_anti_collapse` formerly depended on `entropy_mixture_concavity_axiom`. That axiom has been *removed and promoted to a theorem*, `entropy_mixture_concavity`, proved from primitives via the concave Jensen inequality (`ConcaveOn.le_map_sum`) applied to `Real.concaveOn_negMulLog`, with the pointwise mixture-kernel evaluation `mixtureKernel_apply_toReal` and the finite mass identity `sum_toReal_eq_one`. B3b is now a full theorem with no axiom dependence.

Theorem 27 (B3: mixture lower bound). *`B3_mixture_lb`. Assume Y and Z are finite. Let $\pi : \text{PMF}(Y)$. If the support of π is feasible, y_{low} and y_{high} are feasible, $H_q(y_{\text{low}}) < H_q(y_{\text{high}})$, $\pi(y_{\text{high}}) > 0$, and*

$$R_B(q, y_{\text{low}}) \leq \pi(y_{\text{high}})R_B(q, y_{\text{high}}),$$

then

$$R_B(q, y_{\text{low}}) \leq \sum_y \pi(y)R_B(q, y).$$

Proof. Each term $\pi(y)R_B(q, y)$ in the mixture expectation is nonnegative, so the full finite sum is at least the mass-weighted contribution from y_{high} :

$$\pi(y_{\text{high}})R_B(q, y_{\text{high}}) \leq \sum_y \pi(y)R_B(q, y).$$

The hypothesis $R_B(q, y_{\text{low}}) \leq \pi(y_{\text{high}})R_B(q, y_{\text{high}})$ then gives the claim by transitivity. In Lean, this is obtained from `B3_weighted_high_lb` (Appendix D). $\square$

Remark 1 (Why the mass-weighted assumption is necessary). The stronger informal statement “positive mass on a higher-entropy feasible output suffices” is false in general. A positive probability can be arbitrarily small, so $\pi(y_{\text{high}})R_B(q, y_{\text{high}})$ may still be less than $R_B(q, y_{\text{low}})$. The Lean theorem therefore records the weakest sufficient algebraic condition used by the proof.

Theorem 28 (B3b: Jensen anti-collapse). *`B3b_jensen_anti_collapse` via `entropy_mixture_concavity` (PROOF-09, promoted from axiom). For every prompt q and mixture $\pi : \text{PMF}(Y)$,*

$$\sum_y \pi(y)H_q(y) \leq H(\mu_q(\pi)).$$

Proof. Write both sides as sums of `negMulLog` terms (`shannonEntropy_eq_sum_negMulLog`). The mixture kernel evaluates pointwise to the π -average of component masses (`mixtureKernel_apply_toReal`, from `PMF.bind_apply`). For each downstream state z , apply concave Jensen (`ConcaveOn.le_map_sum` with `Real.concaveOn_negMulLog`), using weights $\pi(y)$ that sum to one (`sum_toReal_eq_one`). Summing over z and interchanging the finite double sum gives the claim. $\square$

Consequence: anti-collapse is not only a pairwise dominance property (B1–B2). At the reward level, if the high-entropy output receives sufficient probability mass, mixing over feasible outputs cannot do worse than collapsing to a strictly dominated lower-entropy output. At the distributional level, B3b states that the entropy of a downstream mixture kernel is at least the π -weighted average of component entropies—now proved from primitives, with no axiom dependence.

7.4 C: Utility Gate, Copy Penalty, and Anti-Padding (Composability Score Only)

The C1–C3 theorems below remain machine-checked but, after R_C removal, apply to the Composability Score only; the Benchmark Score contains no R_C term.

Theorem 29 (C1: utility gate). *C1_utility_gate. For any type α with an `AbstractCompressor`, strings $s, \mathcal{H}$, utility value u , and threshold τ , if*

$$u < \tau,$$

then

$$R_C(s, \mathcal{H}, u, \tau) = 0.$$

Proof. The definition of R_C branches on the condition $\tau \leq u$. The hypothesis $u < \tau$ is equivalent to $\neg(\tau \leq u)$, so the false branch of the definition applies and the reward is 0. Lean proves this by simplifying `rewardC` with `not_le.mpr`. $\square$

Consequence: novelty alone cannot score; infeasible randomness receives zero C reward.

Theorem 30 (C2: copy penalty). *C2_copy_penalty. Under `AbstractCompressor`, for every corpus $\mathcal{H}$,*

$$N(\mathcal{H} \mid \mathcal{H}) \leq L(\mathcal{H}).$$

Proof. Unfolding novelty gives

$$N(\mathcal{H} \mid \mathcal{H}) = L(\mathcal{H} ++ \mathcal{H}) - L(\mathcal{H}).$$

The copy-minimality axiom, applied with $s = \mathcal{H}$ and corpus $\mathcal{H}$, states exactly

$$L(\mathcal{H} ++ \mathcal{H}) - L(\mathcal{H}) \leq L(\mathcal{H}).$$

Thus the desired inequality follows directly. Lean unfolds `novelty` and uses the copy-minimality field of `AbstractCompressor`. $\square$

Consequence: exact corpus copies are bounded by the copy-minimality axiom and cannot be treated as unbounded novelty.

Theorem 31 (C3: anti-padding). *C3_no_padding_exploit. Under `NormalizedCompressor`, for any string s , padding p , and corpus $\mathcal{H}$,*

$$N(s ++ p \mid \mathcal{H}) \leq N(s \mid \mathcal{H}) + L(p).$$

Proof. Expanding the definition of novelty, the desired inequality becomes

$$L(\mathcal{H} ++ (s ++ p)) - L(\mathcal{H}) \leq (L(\mathcal{H} ++ s) - L(\mathcal{H})) + L(p).$$

After adding $L(\mathcal{H})$ to both sides, this is exactly the normalized compressor padding-bound axiom:

$$L(\mathcal{H} ++ (s ++ p)) \leq L(\mathcal{H} ++ s) + L(p).$$

Lean unfolds `novelty`, rewrites list associativity, and finishes by linear arithmetic from the compressor axiom. $\square$

Corollary 1 (Zero-information padding). *C3_no_trivial_padding_gain.* If additionally $L(p) = 0$, then

$$N(s ++ p \mid \mathcal{H}) \leq N(s \mid \mathcal{H}).$$

Proof. Apply C3:

$$N(s ++ p \mid \mathcal{H}) \leq N(s \mid \mathcal{H}) + L(p).$$

Using the additional assumption $L(p) = 0$, the right-hand side simplifies to $N(s \mid \mathcal{H})$. Lean invokes `C3_no_padding_exploit` and then simplifies with $L(p) = 0$. $\square$

Consequence: the C objective closes the length exploit for padding with zero information content.

In practice, C is estimated via KL divergence between output and corpus token PMFs. This is the tractable surface-level approximation of algorithmic code length; the Lean file proves gating, nonnegativity, and the cross-entropy link $N_{\text{KL}} + H(p_s) \approx N(s \mid \mathcal{H})$ in Appendix B.

8 Baseline Contrast: What B and C Add

The Lean development also proves that B and C improve on natural baselines in the exact directions claimed.

8.1 Utility-Only Baseline

Define a utility-only reward

$$R_U(q, y) := \begin{cases} U(q, y), & \text{Feas}_m(q, y), \\ 0, & \text{otherwise.} \end{cases}$$

Theorem 32 (Utility-only indistinguishability). *BBase_indistinguishable.* Assume Z is finite. If y_1, y_2 are feasible, have equal utility,

$$U(q, y_1) = U(q, y_2),$$

and may differ in downstream entropy, then

$$R_U(q, y_1) = R_U(q, y_2).$$

Proof. Since both outputs are feasible, the utility-only reward unfolds to utility on both sides:

$$R_U(q, y_1) = U(q, y_1), \quad R_U(q, y_2) = U(q, y_2).$$

The hypothesis $U(q, y_1) = U(q, y_2)$ then gives $R_U(q, y_1) = R_U(q, y_2)$. The entropy disequality is deliberately unused: it is the feature the utility-only baseline fails to detect. Lean proves this by simplifying `rewardUtil` under feasibility and equal utility. $\square$

Theorem 33 (B distinguishes what utility-only misses). *BContrast_distinguishes.* Under the same feasibility and equal-utility assumptions, if

$$H_q(y_2) < H_q(y_1),$$

then

$$R_U(q, y_1) = R_U(q, y_2) \quad \text{and} \quad R_B(q, y_2) < R_B(q, y_1).$$

Proof. The first conjunct follows from the previous utility-only argument: feasibility opens the utility gate and equal utility makes the two baseline rewards equal. For the second conjunct, feasibility opens the B gate, so

$$R_B(q, y_i) = H_q(y_i) \quad (i = 1, 2).$$

The entropy hypothesis $H_q(y_2) < H_q(y_1)$ then gives $R_B(q, y_2) < R_B(q, y_1)$ by B1. Lean proves the conjunction by simplifying the first component and applying `B1_dominance` for the second. $\square$

8.2 Novelty-Only Baseline

Define an ungated novelty reward

$$R_N(s, \mathcal{H}) := N(s \mid \mathcal{H}).$$

Theorem 34 (Ungated novelty rewards infeasible novelty). *`CBase_infeasible_can_maximize`. Under `AbstractCompressor`, if $u < \tau$ and $0 < N(s \mid \mathcal{H})$, then*

$$R_N(s, \mathcal{H}) > 0 \quad \text{and} \quad R_C(s, \mathcal{H}, u, \tau) = 0.$$

Proof. By definition, $R_N(s, \mathcal{H}) = N(s \mid \mathcal{H})$, so the assumption $0 < N(s \mid \mathcal{H})$ proves $R_N(s, \mathcal{H}) > 0$. Meanwhile $u < \tau$ means the utility gate in R_C is closed, so R_C evaluates to 0. Lean proves the first conjunct by unfolding `rewardNovOnly` and the second by simplifying `rewardC`. $\square$

Consequence: C adds exactly the missing utility gate to novelty-only scoring.

9 Composability of B, C, and D

The Lean file `Creativity/Training/Composability.lean` formalizes the idea that optimizing B, C, and D jointly creates a Pareto-style objective rather than a single reducible scalar. The objectives are genuinely multi-dimensional: Lean contains explicit finite constructions with distinct B, C, and D maximizers (Appendix C). The main structural fact needed in the paper is that the Pareto frontier is nonempty.

Theorem 35 (Nonempty Pareto frontier). *`paretoFrontier_nonempty`. For any finite nonempty output type Y and any rewards $B, C : Y \rightarrow \mathbb{R}$, there exists some $y : Y$ that is not jointly dominated by any other output.*

Proof. Choose y to be an argmax of the finite function B . Such a maximizer exists because Y is finite and nonempty. By the previous theorem, no other output can strictly dominate it in B while being at least as good in C. Therefore this y lies on the Pareto frontier. Lean obtains the finite maximizer using `Finset.exists_mem_eq_sup`. $\square$

Part III

Benchmark Evaluation Objectives (Frozen, No Gradient Access)

This part collects the benchmark-only quantities: the probe-relative structural influence R_D with its `ProbeCompressor` axioms and full hold-out protocol, the interaction gain G_k with its irreducibility

theorems, the O-information sign bridges, and estimator rank preservation for B, C, and D. None of these quantities may receive gradient access.

Goodhart protection. $\mathcal{P}$ is frozen before all experimental conditions (Appendix A hold-out protocol). G_k is computed post-hoc on fixed agent configurations. Neither $\mathcal{P}$ nor any benchmark prompt appears in the training data or reward computation. Violation of this constraint invalidates all rank preservation guarantees.

10 Component D: Probe-Relative Structural Influence

The file `Creativity/D/Deformation.lean` extends the compressor with `ProbeCompressor`, adding two axioms:

- **Incorporation helps:** $N(s' \mid \mathcal{H} \cup \{s\}) \leq N(s' \mid \mathcal{H})$ for every probe s' ;
- **Self-append invariance:** $N(s' \mid \mathcal{H} ++ \mathcal{H}) = N(s' \mid \mathcal{H})$, so exact corpus copies change nothing.

The Lean parameter `probes : List (List  $\alpha$ )` is a fixed finite probe set $\mathcal{P}$ satisfying the benchmark hold-out protocol (Appendix A). The deformation gain is

$$D(s, \mathcal{H}, \mathcal{P}) := \sum_{s' \in \mathcal{P}} (N(s' \mid \mathcal{H}) - N(s' \mid \mathcal{H} \cup \{s\})),$$

formalized as `corpusDeformationGain`. The gated reward is `rewardD`. All D theorems are stated for an arbitrary fixed finite $\mathcal{P}$; they certify algebraic properties of D given $\mathcal{P}$, not the choice of $\mathcal{P}$ itself.

Claim framing. D measures probe-relative structural influence, not transformational creativity *by definition*. A random or unrepresentative $\mathcal{P}$ yields uninformative D . When $\mathcal{P}$ is domain-representative—as under the hold-out protocol below—high D correlates with transformational creativity in Boden’s sense: outputs that compress future domain strings better are restructuring the domain’s information landscape.

Operational probe-set protocol. Construct $\mathcal{P}$ as a stratified hold-out sample from the benchmark task distribution: withhold 10–20% of the evaluation set before any training run, then sample $|\mathcal{P}| = 100\text{--}200$ strings from that hold-out stratified by task category. Freeze $\mathcal{P}$ before all experimental conditions. Record the sampling seed and split index in Appendix A.

Theorem 36 (D1: utility gate). *D1_utility_gate.* If $u < \tau$, then $R_D(q, s) = 0$.

Proof. Same utility-gating argument as C1 and N1. Lean simplifies `rewardD` with `not.le.mpr`. $\square$

Theorem 37 (D2: deformation ordering). *D2_dominance.* If $u \geq \tau$ and $D(s_1, \mathcal{H}, \mathcal{P}) < D(s_2, \mathcal{H}, \mathcal{P})$, then $R_D(q, s_1) < R_D(q, s_2)$.

Proof. Under feasibility, R_D equals the ungated deformation gain. The strict inequality on gains transfers directly. Lean simplifies `rewardD` with the feasibility hypothesis and applies the deformation gap. $\square$

Theorem 38 (D3: exploration bounded by code length (Composability Score only)). *D3_rewardC_bounded.* If $u \geq \tau$, then

$$R_C(q, s) \leq L(s).$$

Proof. Under feasibility, $R_C = N(s \mid \mathcal{H})$. The copy-minimality axiom of `AbstractCompressor` gives exactly $N(s \mid \mathcal{H}) \leq L(s)$. Lean unfolds `rewardC` and applies `copy_minimal`. $\square$

Theorem 39 (D3: joint exploration–transformation bound (Composability Score only)). *D3_exploration_transformation_bound. Under ProbeCompressor, if $u \geq \tau$, then*

$$R_C(q, s) + R_D(q, s) \leq L(s) + D(s, \mathcal{H}, \mathcal{P}).$$

Proof. Under feasibility, the left side is $N(s \mid \mathcal{H}) + D(s, \mathcal{H}, \mathcal{P})$. Apply `D3_rewardC_bounded` for the first term and add the nonnegative second term. Lean rewrites both gated rewards and uses `linarith`. $\square$

Theorem 40 (D3 standalone: R_C -free deformation bound). *D3_rewardD_bound_standalone. Under ProbeCompressor, if $u \geq \tau$, then*

$$R_D(q, s) \leq D(s, \mathcal{H}, \mathcal{P}).$$

This is the D3 bound used by the Benchmark Score: no compression-length ($L(s)/R_C$) term appears.

Proof. Under feasibility, R_D unfolds to the deformation gain itself, so the bound holds with equality. Lean simplifies `rewardD` with the feasibility hypothesis. $\square$

Theorem 41 (D4: zero probe influence for corpus copies). *D4_zero_for_corpus_copy. When $s = \mathcal{H}$ is appended to itself,*

$$D(\mathcal{H}, \mathcal{H}, \mathcal{P}) = 0.$$

Proof. Each probe contribution is $N(s' \mid \mathcal{H}) - N(s' \mid \mathcal{H} ++ \mathcal{H})$. The `append_self_invariant` axiom makes every term zero. Lean proves this by induction on the finite probe list. $\square$

Corollary 2 (Zero gated reward for corpus copy). *D4_zero_reward_for_corpus_copy. If $u \geq \tau$ and $s = \mathcal{H}$, then $R_D(q, s) = 0$.*

Proof. Combine D4 with the feasibility branch of `rewardD`. $\square$

Interpretation: exploratory novelty (C) measures departure from the *current* corpus. Probe-relative structural influence (D) measures how much the output *changes* compression of a fixed hold-out probe set $\mathcal{P}$. High C with $D = 0$ is exploratory creativity within fixed rules. High D on a domain-representative $\mathcal{P}$ correlates with Boden–Wiggins transformational creativity; the Lean theorems do not equate D with transformation outright.

11 Benchmark Estimator Guarantees

The objective theorems are exact. A benchmark only observes estimates. The Lean bridge proves that bounded-error estimators preserve rankings when true gaps exceed twice the error bound.

11.1 Entropy Estimator

Definition 3 (B estimator). *BEstimator.* For a model m , a B estimator consists of

$$\hat{H} : Q \rightarrow Y \rightarrow \mathbb{R}, \quad \epsilon \in \mathbb{R},$$

with uniform error bound

$$|\hat{H}(q, y) - H_q(y)| \leq \epsilon \quad \text{for all } q, y.$$

Theorem 42 (Entropy rank preservation). *B_rank_preserved.* Assume Z is finite. If y_1, y_2 are feasible and

$$H_q(y_2) + 2\epsilon < H_q(y_1),$$

then

$$\hat{H}(q, y_2) < \hat{H}(q, y_1).$$

Proof. The estimator guarantees give

$$|\hat{H}(q, y_i) - H_q(y_i)| \leq \epsilon \quad (i = 1, 2).$$

Thus

$$\hat{H}(q, y_2) \leq H_q(y_2) + \epsilon \quad \text{and} \quad H_q(y_1) - \epsilon \leq \hat{H}(q, y_1).$$

The true-gap assumption

$$H_q(y_2) + 2\epsilon < H_q(y_1)$$

rearranges to

$$H_q(y_2) + \epsilon < H_q(y_1) - \epsilon.$$

Chaining these inequalities yields

$$\hat{H}(q, y_2) \leq H_q(y_2) + \epsilon < H_q(y_1) - \epsilon \leq \hat{H}(q, y_1).$$

Lean extracts the one-sided bounds from `abs_1e.mp` and finishes with `linarith`. □

11.2 Novelty Estimator

Definition 4 (C estimator). *CEstimator.* A C estimator consists of

$$\hat{N} : \text{List}(\alpha) \rightarrow \text{List}(\alpha) \rightarrow \mathbb{R}, \quad \epsilon \in \mathbb{R},$$

with uniform error bound

$$|\hat{N}(s, \mathcal{H}) - N(s \mid \mathcal{H})| \leq \epsilon.$$

Theorem 43 (Novelty rank preservation). *C_rank_preserved.* If

$$N(s_2 \mid \mathcal{H}) + 2\epsilon < N(s_1 \mid \mathcal{H}),$$

then

$$\hat{N}(s_2, \mathcal{H}) < \hat{N}(s_1, \mathcal{H}).$$

Proof. The proof is the novelty analogue of the entropy estimator proof. From the uniform error bound,

$$\widehat{N}(s_2, \mathcal{H}) \leq N(s_2 \mid \mathcal{H}) + \epsilon$$

and

$$N(s_1 \mid \mathcal{H}) - \epsilon \leq \widehat{N}(s_1, \mathcal{H}).$$

The true novelty gap

$$N(s_2 \mid \mathcal{H}) + 2\epsilon < N(s_1 \mid \mathcal{H})$$

implies

$$N(s_2 \mid \mathcal{H}) + \epsilon < N(s_1 \mid \mathcal{H}) - \epsilon.$$

Combining these inequalities gives $\widehat{N}(s_2, \mathcal{H}) < \widehat{N}(s_1, \mathcal{H})$. Lean again uses `abs_le.mp` followed by `linarith`. $\square$

Benchmark implication: the harness is reliable for pairwise ranking only when the true gap exceeds 2ϵ . This is an explicit measurement-resolution condition, not a hidden assumption.

11.3 Proxy Rank Preservation for D

The D component deliberately has no trainable estimator of its own: R_D is benchmark-only. Its rank-preservation analogue is the conditional proxy bridge `D_tilde_proxy_rank_agreement` (Part VI): under the `ProxyFaithfulness` assumption, a proxy gap

$$\tilde{R}_D(s_1) - \tilde{R}_D(s_2) > 2\epsilon$$

forces $R_D(s_1, \mathcal{H}, \mathcal{P}) > R_D(s_2, \mathcal{H}, \mathcal{P})$. Unlike the purely algebraic B and C estimator bounds above, this bridge carries an *empirical* faithfulness condition that must be monitored at every training checkpoint (Part V, Section 23).

12 Multi-Agent Irreducibility

The two-agent Lean model has individual kernels

$$T_1(\cdot \mid q, y_1), \quad T_2(\cdot \mid q, y_2)$$

and a joint kernel

$$T_J(\cdot \mid q, y_1, y_2).$$

It defines

$$H_1 = H(T_1), \quad H_2 = H(T_2), \quad H_J = H(T_J),$$

and the interaction gain

$$G(q, y_1, y_2) := H_J(q, y_1, y_2) - \max\{H_1(q, y_1), H_2(q, y_2)\}.$$

Downstream entropy connects to emergence because it measures whether a system-level output expands the future state space rather than constraining subsequent agents into a narrow trajectory. In O-information language, redundancy repeats the same information across agents, while synergy encodes information no agent holds alone; B rewards outputs that preserve the possibility of such recombination.

Assumption 1 (Faithfulness). *Every directed edge (i, j) in the communication graph $\mathcal{G}$ corresponds to a nonzero pairwise information signal in the joint downstream process: for any edge (i, j) , there exist $q, \mathbf{y}$ such that*

$$I(z_i; z_j \mid q, \mathbf{y}) > 0.$$

This is the standard causal faithfulness assumption (Spirtes, Glymour, and Scheines, 2000; Pearl, 2009). It rules out degenerate parameter settings in which agent j structurally depends on agent i but the statistical effect cancels to zero. Operationally, faithfulness is testable in the benchmark by measuring pairwise TDMI between agent message streams; TDMI above the 2ϵ resolution threshold validates the assumption on the observed task distribution.

Theorem 44 (Corrected zero-gain characterization (PROOF-11)). *`gain_zero_iff_joint_eq_max`. For the scalar entropy layer,*

$$G_k(q, \mathbf{y}) = 0 \iff H_J(q, \mathbf{y}) = \max_i H_i(q, y_i),$$

i.e. zero gain holds exactly in the degenerate case where the joint process adds nothing over the best individual agent.

Known gap resolved: the old bridge axiom was false. The previous axiom `gain_zero_iff_productKernel` asserted $G_k = 0 \iff \text{isProductKernel}(m)$. This is *incorrect* for product kernels with positive marginals: entropy additivity (`productKernel_joint_entropy_eq_sum`, now a theorem, PROOF-10) gives $H_J = H_1 + H_2 > \max(H_1, H_2)$ whenever both marginal entropies are positive, so the gain is strictly *positive*, not zero. The axiom has been removed from the Lean development. The refuting direction is machine-checked as `productKernel_additivity_gain_pos`: for a two-agent coupling with additive joint entropy and positive marginals, $0 < G_k$. The corrected bi-conditional `gain_biconditional` is stated against the degeneracy condition $H_J = \max_i H_i$ and is proved, not assumed.

Theorem 45 (Irreducibility from positive interaction gain). *`MAS_irreducibility`. Assume Z is finite. If*

$$0 < G(q, y_1, y_2),$$

then

$$H_1(q, y_1) < H_J(q, y_1, y_2) \quad \text{and} \quad H_2(q, y_2) < H_J(q, y_1, y_2).$$

Proof. Expanding interaction gain gives

$$0 < H_J(q, y_1, y_2) - \max\{H_1(q, y_1), H_2(q, y_2)\}.$$

Equivalently,

$$\max\{H_1(q, y_1), H_2(q, y_2)\} < H_J(q, y_1, y_2).$$

But each individual entropy is bounded above by the maximum:

$$H_1(q, y_1) \leq \max\{H_1(q, y_1), H_2(q, y_2)\},$$

$$H_2(q, y_2) \leq \max\{H_1(q, y_1), H_2(q, y_2)\}.$$

Transitivity gives both strict inequalities against H_J . Lean unfolds `interactionGain` and uses `le_max_left`, `le_max_right`, and `linarith`. $\square$

Consequence: positive interaction gain formally certifies that neither agent alone achieves the joint downstream entropy. Under Assumption 1 and the product-kernel bridge below, positive G reflects genuine multi-agent coordination: zero gain corresponds to product-kernel independence, while positive gain corresponds to a non-product joint downstream process.

Definition 5 (Two-agent estimator). *TwoAgentEstimator*. The MAS benchmark estimator supplies

$$\hat{H}_1, \quad \hat{H}_2, \quad \hat{H}_J$$

with a common uniform error bound ϵ :

$$|\hat{H}_1 - H_1| \leq \epsilon, \quad |\hat{H}_2 - H_2| \leq \epsilon, \quad |\hat{H}_J - H_J| \leq \epsilon.$$

Theorem 46 (Benchmark detection of irreducibility). *MAS_benchmark_detects_irreducibility*. If

$$2\epsilon < G(q, y_1, y_2),$$

then

$$\max\{\hat{H}_1(q, y_1), \hat{H}_2(q, y_2)\} < \hat{H}_J(q, y_1, y_2).$$

Proof. The estimator bounds imply

$$\hat{H}_1(q, y_1) \leq H_1(q, y_1) + \epsilon, \quad \hat{H}_2(q, y_2) \leq H_2(q, y_2) + \epsilon,$$

and

$$H_J(q, y_1, y_2) - \epsilon \leq \hat{H}_J(q, y_1, y_2).$$

The gain condition

$$2\epsilon < H_J(q, y_1, y_2) - \max\{H_1(q, y_1), H_2(q, y_2)\}$$

rearranges to

$$\max\{H_1(q, y_1), H_2(q, y_2)\} + \epsilon < H_J(q, y_1, y_2) - \epsilon.$$

Since H_1 and H_2 are each at most their maximum, the preceding inequalities give

$$\hat{H}_1(q, y_1) < \hat{H}_J(q, y_1, y_2) \quad \text{and} \quad \hat{H}_2(q, y_2) < \hat{H}_J(q, y_1, y_2).$$

The characterization

$$\max\{a, b\} < c \iff a < c \text{ and } b < c$$

then yields the desired result. Lean extracts the one-sided bounds with `abs_le.mp`, rewrites by `max_lt_iff`, and solves both goals with `linarith`. $\square$

Benchmark implication: when interaction gain exceeds twice the estimator error, the harness detects the joint behavior as irreducible.

12.1 Extension to k -Agent Systems

The two-agent theorem is the smallest case of a more general Lean development. For $k > 0$ agents, the project defines a `KAgentModel` with a communication graph `CommGraph`. A graph edge

$$\mathcal{G}(i, j)$$

means that agent j receives information from agent i . The model also records an abstract pairwise information quantity `pairMutualInfo` and a faithfulness field: every edge has some task/output witness with strictly positive pairwise information. The individual kernels

$$T_i(\cdot \mid q, y_i) \quad (i : \text{Fin}(k))$$

and a joint kernel

$$T_J(\cdot \mid q, \mathbf{y}), \quad \mathbf{y} : \text{Fin}(k) \rightarrow Y.$$

The individual and joint entropies are

$$H_i(q, y_i) := H(T_i(\cdot \mid q, y_i)), \quad H_J(q, \mathbf{y}) := H(T_J(\cdot \mid q, \mathbf{y})).$$

The k -agent interaction gain is

$$G_k(q, \mathbf{y}) := H_J(q, \mathbf{y}) - \max_{i : \text{Fin}(k)} H_i(q, y_i).$$

This is the direct generalization of the two-agent gain: it compares the joint downstream entropy to the strongest individual downstream entropy among all agents.

Theorem 47 (Base non-product kernel). *`nonProduct_base_case`. For two agents, if the communication graph has an edge $0 \rightarrow 1$, then faithfulness rules out product-kernel independence:*

$$\neg \text{isProductKernel}(m).$$

Proof. Faithfulness supplies a witness $q, \mathbf{y}$ with positive pairwise information along the edge. Product-kernel independence forces all off-diagonal pairwise information values to be zero. These two statements contradict each other. Lean closes the contradiction by rewriting the product consequence and using `linarith`. $\square$

Theorem 48 (Non-product kernel from an active graph). *`connectedMAS_nonProductKernel`. For any number of agents, if `CommGraph` contains at least one edge and the model is faithful, then*

$$\neg \text{isProductKernel}(m).$$

Proof. Choose one edge (i, j) . Irreflexivity gives $i \neq j$. Faithfulness gives positive pairwise information for that edge at some $q, \mathbf{y}$. A product kernel forces the same pairwise information value to be zero, contradiction. No induction on k is needed: one faithful edge is enough. $\square$

Theorem 49 (Gain biconditional, corrected). *`gain_biconditional`. If $0 \leq G_k(q, \mathbf{y})$, then*

$$G_k(q, \mathbf{y}) = 0 \iff H_J(q, \mathbf{y}) = \max_i H_i(q, y_i),$$

and

$$0 < G_k(q, \mathbf{y}) \iff H_J(q, \mathbf{y}) \neq \max_i H_i(q, y_i).$$

Proof. The first equivalence is `gain_zero_iff_joint_eq_max`, an algebraic identity on $G_k = H_J - \max_i H_i$. For the second, nonnegativity leaves only two cases: $G_k = 0$ or $0 < G_k$; the zero case is equivalent to the degeneracy condition, so non-degeneracy is equivalent to strict positivity. The former product-kernel version of this biconditional relied on the removed incorrect axiom; see `productKernel_additivity_gain_pos` for the refutation. $\square$

Theorem 50 (*k*-agent irreducibility). *kAgent_irreducibility*. Assume Z is finite and m is a *KAgentModel* with $0 < k$. If

$$0 < G_k(q, \mathbf{y}),$$

then for every agent $i : \text{Fin}(k)$,

$$H_i(q, y_i) < H_J(q, \mathbf{y}).$$

Proof. Expanding the definition of G_k gives

$$0 < H_J(q, \mathbf{y}) - \max_{i:\text{Fin}(k)} H_i(q, y_i),$$

so

$$\max_{i:\text{Fin}(k)} H_i(q, y_i) < H_J(q, \mathbf{y}).$$

For each fixed agent i , its individual entropy is bounded by the finite maximum:

$$H_i(q, y_i) \leq \max_{j:\text{Fin}(k)} H_j(q, y_j).$$

Transitivity gives

$$H_i(q, y_i) < H_J(q, \mathbf{y})$$

for every i . Lean proves this using the finite supremum lemma `Finset.le_sup'` and the definition of `kInteractionGain`. $\square$

Definition 6 (*k*-agent estimator). *KAgentEstimator*. For a fixed model m , prompt q , and joint action $\mathbf{y}$, the estimator supplies individual estimates $\hat{H}_i$, a joint estimate $\hat{H}_J$, and a common error bound ϵ such that

$$|\hat{H}_i - H_i(q, y_i)| \leq \epsilon \quad \text{for all } i, \quad |\hat{H}_J - H_J(q, \mathbf{y})| \leq \epsilon.$$

Theorem 51 (*k*-agent benchmark detection). *kAgent_benchmark_detects*. If

$$2\epsilon < G_k(q, \mathbf{y}),$$

then

$$\max_{i:\text{Fin}(k)} \hat{H}_i < \hat{H}_J.$$

Proof. The proof is the same measurement-resolution argument as in the two-agent case. For every i ,

$$\hat{H}_i \leq H_i(q, y_i) + \epsilon,$$

while

$$H_J(q, \mathbf{y}) - \epsilon \leq \hat{H}_J.$$

The assumption $2\epsilon < G_k(q, \mathbf{y})$ expands to

$$\max_i H_i(q, y_i) + \epsilon < H_J(q, \mathbf{y}) - \epsilon.$$

Since each $H_i(q, y_i)$ is at most the maximum, each estimate $\hat{H}_i$ is strictly below $\hat{H}_J$. Therefore the finite maximum of the individual estimates is also strictly below $\hat{H}_J$. Lean proves this by rewriting with `Finset.sup'_lt_iff`, extracting estimator bounds using `abs_le.mp`, and closing each agent case with `linarith`. $\square$

Thus the formal irreducibility criterion is not restricted to two agents. The two-agent theorem is the readable base case; the *k*-agent theorem is the general MAS benchmark contract.

12.2 O-Information Sign Bridges

The Lean file `Creativity/MAS/OInformation.lean` also includes scalar O-information sign checks for the three-agent case. The simplified quantity is

$$\Omega_3(H_{12}, H_{13}, H_{23}, H_{123}) := H_{12} + H_{13} + H_{23} - H_{123}.$$

Theorem 52 (Negative O-information from pair-sum bound). *oInformation3_negative_of_pair_sum_lt.* If

$$H_{12} + H_{13} + H_{23} < H_{123},$$

then

$$\Omega_3(H_{12}, H_{13}, H_{23}, H_{123}) < 0.$$

Proof. Unfolding Ω_3 turns the conclusion into $H_{12} + H_{13} + H_{23} - H_{123} < 0$, which is exactly the assumed strict inequality rearranged. Lean unfolds `oInformation3` and uses `linarith`. $\square$

Theorem 53 (Positive O-information from triple-below-pair-sum). *oInformation3_positive_of_triple_lt_pair_sum.* If

$$H_{123} < H_{12} + H_{13} + H_{23},$$

then

$$0 < \Omega_3(H_{12}, H_{13}, H_{23}, H_{123}).$$

Proof. After unfolding Ω_3 , the conclusion is $0 < H_{12} + H_{13} + H_{23} - H_{123}$, which follows by rearranging the hypothesis. Lean again closes the arithmetic by `linarith`. $\square$

Theorem 54 (Equal-marginals synergy bridge). *equalMarginals_gain_implies_synergy.* If $h < H_J$ and the additional sign condition

$$3(H_J - h) < H_J$$

holds, then

$$\Omega_3(H_J - h, H_J - h, H_J - h, H_J) < 0.$$

Proof. Substituting the equal-marginals expression into Ω_3 gives

$$3(H_J - h) - H_J.$$

The sign condition says this quantity is negative. Lean unfolds `oInformation3` and uses `linarith`. The explicit sign condition is necessary: $h < H_J$ alone does not force $3(H_J - h) < H_J$. $\square$

Theorem 55 (Negative-gain redundancy bridge). *negativeGain_implies_redundancy.* If $H_J < h$ and

$$H_J < 3(h - H_J),$$

then

$$0 < \Omega_3(h - H_J, h - H_J, h - H_J, H_J).$$

Proof. Unfolding Ω_3 reduces the conclusion to $0 < 3(h - H_J) - H_J$, which is exactly the second hypothesis rearranged. Lean finishes by `linarith`. $\square$

12.3 O-Information Kernel Bridge

The file `Creativity/MAS/OInfoBridge.lean` connects the scalar O-information sign lemmas to concrete product-kernel structure. A `TwoAgentProductModel` specifies independent kernels k_1, k_2 and forms their joint PMF by independent coupling.

Theorem 56 (Product-kernel entropy additivity (PROOF-10, promoted from axiom)). *`productKernel_joint_entropy_eq_sum`. For finite downstream alphabet Z and independent product coupling,*

$$H(Z_1, Z_2) = H(Z_1) + H(Z_2),$$

where (Z_1, Z_2) are drawn from the product kernel induced by $k_1(q, y_1)$ and $k_2(q, y_2)$.

Proof. Formerly an axiom; now proved as `shannonEntropy_jointPMF`. The product coupling factorizes pointwise, $(p \otimes q)(a, b) = p(a)q(b)$ (`jointPMF_apply`, via `PMF.bind_apply` and `PMF.map_apply`). Expanding the joint entropy over $Z \times Z$ and applying the logarithm product rule in the form `negMulLog(xy) = y negMulLog(x) + x negMulLog(y)` (`Real.negMulLog_mul`) splits the double sum into $\sum_a \text{negMulLog}(p(a)) \sum_b q(b) + \sum_b \text{negMulLog}(q(b)) \sum_a p(a)$; the mass identities $\sum p = \sum q = 1$ (`sum_toReal_eq_one`) reduce this to $H(Z_1) + H(Z_2)$. $\square$

Theorem 57 (Product kernels yield positive pair-sum entropy). *`productKernel_implies_redundancy`. If $0 < H(k_1(q, y_1))$ and $0 < H(k_2(q, y_2))$, then*

$$0 < H(k_1(q, y_1)) + H(k_2(q, y_2)).$$

Proof. This is pure arithmetic: the sum of two strictly positive reals is strictly positive. Lean closes with `linarith`. $\square$

Theorem 58 (Product-kernel gain is not synergy). *`productKernel_gain_is_not_synergy`. Independent product kernels can have positive individual and joint entropies without satisfying a negative O-information condition. The Lean development records this as a scoped boundary theorem: product independence is not the same object as irreducible synergy.*

Proof. Lean proves this documentation theorem by `trivial`. The mathematical content is the conceptual separation between product-kernel additivity and the sign conditions required for negative O-information in the scalar bridge lemmas above. $\square$

13 Interaction Gain to Decision Quality: the CUE Bridge

The entropy-level irreducibility theorems above certify that positive G_k means no single agent reproduces the joint downstream process. The following bridge (PROOF-07) lifts this to the receiver decision layer of Part I.

Definition 7 (Receiver calibration link). *`ReceiverCalibratedMAS`. A receiver whose CUE readings are calibrated to downstream entropy with uniform error ε_{cal} : for each agent i , $|\text{CUE}_i - H_i| \leq \varepsilon_{\text{cal}}$, and for the joint output, $|\text{CUE}_J - H_J| \leq \varepsilon_{\text{cal}}$. This is the formal `brier_entropy_calibration_link`, parallel in structure to `KAgentEstimator`.*

Theorem 59 (PROOF-07: interaction gain forces CUE improvement). *`Gk_implies_CUE_improvement` and `Gk_implies_CUE_improvement_sup`. Under the receiver calibration link, if*

$$2\varepsilon_{\text{cal}} < G_k(q, \mathbf{y}),$$

then $\text{CUE}_i < \text{CUE}_J$ for every agent i , and hence

$$\max_i \text{CUE}(y_i, A, T) < \text{CUE}(y_{\text{joint}}, A, T).$$

Proof. The CUE analogue of `kAgent_benchmark_detects`: each agent CUE is at most $H_i + \varepsilon_{\text{cal}} \leq \max_i H_i + \varepsilon_{\text{cal}}$, the joint CUE is at least $H_J - \varepsilon_{\text{cal}}$, and the gain hypothesis separates the two bounds. Lean closes with `linarith` over the absolute-value bounds. $\square$

14 Trajectory-Level Diagnostics: Step-CUE and Diverge-Converge

These benchmark diagnostics evaluate the *reasoning trajectory* $y_{1:t}$ rather than only the final output.

Theorem 60 (PROOF-04: gated Step-CUE curves are monotone). *`CUE_trajectory_monotone_nondecreasing`. Let $C(t) = \text{CUE}(y_{1:t}, A, T)$ be the running Step-CUE curve and $r_t = C(t+1) - C(t)$ the incremental step reward (`stepReward`). If every step satisfies the utility gate ($r_t \geq 0$ for all t), then C is non-decreasing.*

Proof. Nonnegative successive differences give monotonicity; `monotone_nat_of_le_succ` plus `linarith`. $\square$

Theorem 61 (PROOF-14: γ -positivity characterization). *`step_CUE_gamma_positivity_characterization`. For the fitted Step-CUE curve $C(t) = C_\infty(1 - e^{-\mu t}) + \gamma t$ (`stepCUEFit`), the linear saturation rate satisfies*

$$\gamma > 0 \iff \exists t > 0 : C_\infty(1 - e^{-\mu t}) < C(t),$$

i.e. γ is positive exactly when some step exceeds the pure exponential-saturation prediction — genuine continuous discovery.

Proof. The fitted curve exceeds its exponential component by exactly γt ; positivity of γt at a positive time is equivalent to positivity of γ . Lean closes both directions with `linarith/nlinarith`. $\square$

Theorem 62 (PROOF-05: Diverge-Converge score existence and non-triviality). *`DC_score_existence` and `monotone_increasing_not_DC`. There exists a receiver-entropy trajectory E and a peak index t^* satisfying the diverge-converge predicate (`hasDivergeConverge`): E strictly increases up to t^* (divergence) and strictly decreases afterward (convergence), so $\text{DC} = 1$ is achievable. Conversely, any strictly increasing (monotone-divergent) trajectory fails the predicate for every choice of t^* , so $\text{DC} = 0$ on monotone trajectories: the score is neither trivially satisfied nor trivially failed.*

Proof. Existence is by explicit construction: the tent-shaped trajectory $E(t) = 1 - (t - 1)^2$ on three steps peaks at $t^* = 1$ (`tentTrajectory`). Non-triviality: the convergence phase at the peak would contradict monotonicity at t^* ; `linarith`. $\square$

15 Estimator Calibration and Benchmark Specification

The Lean benchmark layer contains one imported statistical axiom and one machine-readable experiment specification. The plug-in entropy estimator is defined directly from empirical counts:

$$\hat{H}_{\text{plug}}(c, n) = - \sum_y \frac{c_y}{n} \log \frac{c_y}{n}.$$

Assumption 2 (Plug-in entropy bias bound (PROOF-12; Known Gaps Registry KG-2)). *plugIn_bias_axiom*. For alphabet Y , counts $c : Y \rightarrow \mathbb{N}$, sample size $n > 0$, and true entropy H , the imported Miller–Madow-scale bound is

$$\left| \hat{H}_{\text{plug}}(c, n) - H \right| \leq \frac{|Y|}{2n}.$$

*This is a registered trusted import (Miller & Madow, 1955), not an internally proved theorem. Proof target for future promotion: Hoeffding’s inequality applied to the empirical frequency estimator, combined with the Lipschitz continuity of $x \mapsto -x \log x$ on $[0, 1]$ (each term bounded in $[0, 1/e]$); a Lean 4 statistical-learning-theory library providing *Hoeffding-inequality* would discharge it. The subsequent Lean corollary uses the bound algebraically.*

Theorem 63 (Sample requirement for entropy ranking). *entropy_ranking_sample_requirement*. If

$$H_{\text{low}} + \frac{|Y|}{n} < H_{\text{high}},$$

then the plug-in estimates preserve the ordering:

$$\hat{H}_{\text{plug}}(c_{\text{low}}, n) < \hat{H}_{\text{plug}}(c_{\text{high}}, n).$$

Proof. The bias axiom gives each estimate error at most $|Y|/(2n)$. Therefore the total possible two-sided error between the two estimates is at most $|Y|/n$. If the true gap exceeds this amount, the lower estimate remains below the higher estimate even in the worst case. Lean extracts the absolute-value bounds with `abs_le.mp`, proves

$$2 \cdot \frac{|Y|}{2n} = \frac{|Y|}{n},$$

and finishes by `linarith`. □

To make the resolution condition concrete: for a 50-token effective alphabet and $n = 200$ episodes, the calibration gives

$$\epsilon = \frac{|Y|}{2n} = \frac{50}{400} = 0.125$$

nats, so the benchmark’s two-sided detection floor is $2\epsilon = 0.25$ nats. Gaps below 0.25 nats require larger samples; for example, reaching a 0.10-nat floor with the same effective alphabet requires $n \geq 500$ episodes. All reported experiments should state their effective alphabet size, sample count, and minimum observed entropy gap against this calibration floor.

The experiment specification file defines four topology labels `Topology`, an episode record `EpisodeResult`, and two predicates: `predictedS2DominatesS1` and `predictedGroupthinkSignature`.

Theorem 64 (Detectability of S2 dominance). *benchmark_can_detect_S2_dominance*. If the true interaction gain gap between decentralized S2 and dense-centralized S1 exceeds two resolution margins,

$$G_{S1} + 4\epsilon < G_{S2},$$

then there exist benchmark episode records r_2, r_1 with topologies S2 and S1 respectively such that

$$\text{predictedS2DominatesS1}(r_2, r_1)$$

holds.

Proof. Lean constructs worst-case estimated gains:

$$\hat{G}_{S2} = G_{S2} - \epsilon, \quad \hat{G}_{S1} = G_{S1} + \epsilon.$$

The predicate requires

$$\hat{G}_{S1} + 2\epsilon < \hat{G}_{S2},$$

which becomes

$$G_{S1} + 3\epsilon < G_{S2} - \epsilon,$$

equivalently $G_{S1} + 4\epsilon < G_{S2}$. This is exactly the hypothesis. Lean fills the existential records explicitly and closes the arithmetic with `linarith`. $\square$

Benchmark interpretation. The benchmark operationalizes the formal results by checking whether systems optimizing B maintain richer future trajectory spaces, whether systems optimizing C reduce corpus-adjacent repetition without rewarding incoherence or padding, and whether multi-agent systems with detected interaction gain behave irreducibly rather than as sums of individual agents. The theory says what should happen; the benchmark tests whether it happens under the stated resolution and faithfulness assumptions.

Part IV

Dual-Process Separation Principle

This part sits deliberately *between* the frozen benchmark metrics (Part III) and the training signal (Part V): the separation proved here is what makes the benchmark non-Goodhartable. Goodhart resistance in this framework is a **two-level structure**:

- **Level 1 — Architectural Goodhart resistance.** The Dual-Process Separation Principle (PROOF-03): the training gradient is structurally independent of every frozen benchmark object.
- **Level 2 — Protocol Goodhart resistance.** The four protocol conditions collected in `Goodhart_resistance_summary` (Part V), each paired with its collapse direction by the Structural Goodhart Theorem (PROOF-13) below.

16 The Separation Theorem

Let $\Pi_{\text{frozen}} = \{\mathcal{P}, T, A\}$ be the frozen benchmark objects (probe set, task battery, receiver) and $\Pi_{\text{live}} = \{\tilde{R}_D\text{-probes, policy, corpus schedule}\}$ the live training quantities. A training configuration splits into a live and a frozen component (`TrainingConfig`); a reward *depends only on the live component* when any two configurations agreeing on the live part receive equal reward (`dependsOnlyOnLive`) — the discrete statement that its gradient in the frozen coordinate is zero.

Theorem 65 (PROOF-03: Dual-Process Separation Principle). *`dual_process_separation_principle`. Any training reward built as a function of the live component alone (`liveReward`) depends only on the live component: for every gradient update step, $\nabla_{\theta} R_{\text{train}} \perp \Pi_{\text{frozen}}$.*

Corollary 3 (No differentiable Goodhart path). *`Rtrain_cannot_goodhart_frozen`. Changing any frozen benchmark object while holding the live component fixed leaves R_{train} unchanged. Consequently no gradient step on R_{train} can be driven by, or exploit, a frozen object: R_{train} cannot Goodhart the benchmark through any differentiable path. Combined with `trainReward_neq_fullReward`, the train/benchmark split is structural, not merely definitional.*

17 The Structural Goodhart Collapse Theorem

Protection and violation are now a theorem pair: the rank-agreement guarantee (`D_tilde_proxy_rank_agreement`, Part VI) holds under the protocol conditions, and this section proves each violation admits a refuting exploitation path.

Definition 8 (Rank agreement and rank reversal). *`rankAgreement`, `rankReversal`. For a proxy/benchmark instance with resolution margin ε , rank agreement demands that a proxy gap beyond 2ε forces the same benchmark ordering; a rank reversal is a pair s_1, s_2 where the proxy prefers s_1 beyond 2ε while the benchmark strictly prefers s_2 .*

Theorem 66 (PROOF-13: Structural Goodhart Collapse). *`rank_reversal_refutes_agreement`, `reversal_instance_exists`, `structural_goodhart_collapse`. Any rank reversal refutes rank agreement, and rank reversals are realizable: there is an explicit two-candidate instance whose proxy ranks the candidates oppositely to the benchmark beyond the margin. Hence whenever a violated Goodhart condition admits a reversal, the proxy-rank-agreement guarantee collapses.*

Remark 2 (The four exploitation paths). Each protocol condition instantiates the collapse through its own path: (i) *fixed probe target* — if Q_{probe} contains a fixed string across steps, the policy can memorize it, inflating $\tilde{R}_D$ without improving R_D on the frozen $\mathcal{P}$; (ii) *corpus non-stationarity* — if $\mathcal{H}$ absorbs policy outputs, the anti-padding baseline shifts and `C3_no_padding_exploit` loses its precondition; (iii) *loose utility gate* — if $\tau \leq H_q(y_{\text{random}})$, incoherent high-entropy outputs pass the gate (`CBase_infeasible_can_maximize` pattern); (iv) *undetected proxy decay* — if $\rho(\tilde{R}_D, R_D) < \rho_{\min}$ without detection, the faithfulness premise of rank agreement fails. In each case the admitted reversal triggers `rank_reversal_refutes_agreement`.

Part V

2-CGRPO Training Signal (Certified, Goodhart-Resistant)

This part defines the reward the trainer actually optimizes, proves that DPO steps ascend each of its components, and states the four Goodhart-resistance conditions that a training run must satisfy for the certificates to apply.

18 Training Reward Definition

The training reward is

$$R_{\text{train}}(q, y, s) := R_B(q, y) + \lambda_{\tilde{D}} \tilde{R}_D(q, s) \quad (\text{trainReward}),$$

which is a distinct object from the reported multiplicative Benchmark Score $R_{\text{creativity}}$ (1) (`Rcreativity`, Section 4). There is no $\lambda_C R_C$ channel. The components are:

- R_B : utility-gated downstream entropy, exactly as in Part II.
- $\tilde{R}_D$: the trainable proxy for D, defined below on *resampled* probe prompts, never on the frozen benchmark probe set $\mathcal{P}$.

The train/benchmark split is machine-checked by `trainReward_neq_benchmarkReward`.

18.1 The Trainable Proxy $\tilde{R}_D$

The Lean structure `DTildeModel` (`Creativity/D/ProxyReward.lean`) contains a probe prompt distribution $Q_{\text{probe}} : \text{PMF}(Q)$ that is resampled per batch and never frozen, a frozen reference policy $p_{\text{ref}} : Q \rightarrow \text{PMF}(Z)$ that is never updated during training, and the trained policy’s conditioned predictive distribution $p(\cdot \mid q_{\text{probe}}, s)$. The proxy reward is

$$\tilde{R}_D(s, q_{\text{probe}}) = D_{\text{KL}}\left(p(\cdot \mid q_{\text{probe}}, s) \parallel p_{\text{ref}}(\cdot \mid q_{\text{probe}})\right) \cdot \mathbf{1}[u \geq \tau],$$

formalized as `rewardDTilde`.

Theorem 67 ($\tilde{D}$ utility gate). *`D_tilde_utility_gate`. If $u < \tau$, then $\tilde{R}_D(s, q_{\text{probe}}) = 0$.*

Proof. Mirrors `D1_utility_gate`: the gate closes the reward branch. Lean simplifies `rewardDTilde` with `not_le.mpr`. $\square$

Theorem 68 ($\tilde{D}$ nonnegativity). *`D_tilde_nonneg`. For all $s, q_{\text{probe}}, u, \tau$, $0 \leq \tilde{R}_D(s, q_{\text{probe}})$.*

Proof. The gated branch is a KL divergence, nonnegative by `klDivergence_nonneg`; the closed branch is 0. $\square$

Theorem 69 (Zero proxy reward for reference-identical policies). *`D_tilde_zero_for_identical`. If $p(\cdot \mid q_{\text{probe}}, s) = p_{\text{ref}}(\cdot \mid q_{\text{probe}})$, then $\tilde{R}_D(s, q_{\text{probe}}) = 0$.*

Proof. Mirrors `D4_zero_for_corpus_copy`: reproducing the reference distribution earns nothing. Lean rewrites with the identity hypothesis and the term-by-term lemma `klDivergence_self`. $\square$

Assumption 3 (No fixed probe target). *`D_tilde_no_fixed_target`. Q_{probe} is drawn fresh each step: no string in Q_{probe} is fixed across training steps. Formally, for the probe batch schedule of any admissible protocol (`DTildeTrainingProtocol`), no probe prompt belongs to the batch at every step. This is the formal Goodhart-resistance condition for $\tilde{R}_D$: it prevents memorization of specific probe inputs.*

Theorem 70 ($\tilde{D}$ preference ordering). *`D_tilde_dominance`. If $u \geq \tau$ and $\tilde{R}_D(s_1) < \tilde{R}_D(s_2)$, then (s_2, s_1) is a $\tilde{D}$ -valid preference pair.*

Proof. Same structure as `B1_dominance` and `D2_dominance`: under the open gate the reward unfolds to the KL shift, so the strict ordering transfers. This is the prerequisite for `DPO_D_tilde_gradient_correct` below. $\square$

18.2 Properties of R_{train}

Theorem 71 (Training reward nonnegativity). *`trainReward_nonneg`. If $0 \leq R_B$, $0 \leq R_C$, $0 \leq \lambda_C$, and $0 \leq \lambda_{\tilde{D}}$, then $0 \leq R_{\text{train}}(q, y, s)$.*

Proof. Mirrors `benchmarkReward_nonneg`; the $\tilde{D}$ component is nonnegative unconditionally by `D_tilde_nonneg`. $\square$

Theorem 72 (Training advantage sign correctness). *`trainReward_advantage_sign_correct`. For a group of R_{train} scores, if $r_j < \mu < r_i$ and the stabilizer $\epsilon > 0$, then $A_j < 0$ and $0 < A_i$.*

Proof. Mirrors `advantage_sign_correct`, instantiated at the training reward via `trainRewards`. $\square$

Theorem 73 (Train/benchmark split). *`trainReward_neq_fullReward`. $R_{\text{train}} \neq R_{\text{full}}$: whenever $\lambda_{\tilde{D}} \tilde{R}_D(s) \neq \lambda_D R_D(s, \mathcal{H}, \mathcal{P})$, the two rewards differ at that point. R_D and $\tilde{R}_D$ are distinct objects; this theorem exists solely to make the train/benchmark split machine-checkable, not just a prose convention.*

Proof. Both rewards share the B and C terms, so their difference is exactly the difference of the weighted D components. Lean unfolds both definitions and closes with `linarith`. $\square$

19 Training Correspondence: DPO Direction

The Lean development proves a local DPO-B training guarantee at the abstraction level used in the finite model. The calculus-heavy softmax-gradient step is represented by two explicit local hypotheses: **H1** non-degeneracy, so the winner and loser log-probability gradients are not identical, and **H2** a sufficiently small step size, so the local DPO step transfers positive probability mass from the B-loser to the B-winner while leaving other outputs fixed to first order. Under this local transfer hypothesis, Lean proves that expected B reward strictly increases.

For reference, the older bridge file `Creativity/B/DPO.lean` also defines a finite softmax policy `softmaxPolicy` and DPO log-odds $\text{LogOdds}_\ell(y_w, y_l) = \ell_{y_w} - \ell_{y_l}$.

Theorem 74 (Finite logit update increases winner log-odds). *`dpo_logit_direction`. Assume Y has decidable equality. Let $\ell : Y \rightarrow \mathbb{R}$, distinct winner and loser indices $y_w \neq y_l$, and step $\eta > 0$. Define the updated logits*

$$\ell'_y = \begin{cases} \ell_y + \eta, & y = y_w, \\ \ell_y - \eta, & y = y_l, \\ \ell_y, & \text{otherwise.} \end{cases}$$

Then

$$\text{LogOdds}_\ell(y_w, y_l) < \text{LogOdds}_{\ell'}(y_w, y_l).$$

Proof. Because $y_w \neq y_l$, the update gives $\ell'_{y_w} = \ell_{y_w} + \eta$ and $\ell'_{y_l} = \ell_{y_l} - \eta$. Hence

$$\text{LogOdds}_{\ell'}(y_w, y_l) = (\ell_{y_w} + \eta) - (\ell_{y_l} - \eta) = \text{LogOdds}_\ell(y_w, y_l) + 2\eta.$$

Since $\eta > 0$, the strict increase follows. Lean packages the update in `LogitUpdate/applyUpdate`, simplifies with the disequality hypothesis, and closes with `linarith`. $\square$

Theorem 75 (DPO gradient alignment). *dpo_gradient_increases_logodds. If $\beta > 0$, $\sigma(r_\theta(y_l) - r_\theta(y_w)) > 0$, the squared gradient-difference norm is positive, and $\eta > 0$, then the first-order DPO update has strictly positive alignment with the winner-over-loser log-odds direction.*

Proof. Lean formalizes the alignment scalar as

$$\beta \sigma(r_l - r_w) \|\nabla \log \pi_\theta(y_w) - \nabla \log \pi_\theta(y_l)\|^2.$$

All factors are strictly positive under H1 and $\beta > 0$, so multiplying by $\eta > 0$ remains strictly positive. Lean closes this with positivity. $\square$

Theorem 76 (Softmax monotonicity for the winning logit). *softmax_monotone_in_logit_diff. For two competing logits, increasing the winner logit by any $\delta > 0$ strictly increases the winner's softmax probability.*

Proof. Writing $a = \exp(\ell_w) > 0$, $b = \exp(\ell_l) > 0$, and $c = \exp(\delta) > 1$, the claim is

$$\frac{a}{a+b} < \frac{ac}{ac+b}.$$

Cross-multiplication over positive denominators reduces this to $ab < acb$, which follows from $c > 1$. Lean proves the inequality by exponential positivity, cross-multiplication, and ring normalization. $\square$

Theorem 77 (B-valid probability transfer increases expected B). *dpo_step_increases_expected_RB. Let a local DPO step transfer $\delta > 0$ probability mass from y_l to y_w , holding all other output probabilities fixed. If*

$$R_B(q, y_l) < R_B(q, y_w),$$

then the policy expectation of R_B strictly increases.

Proof. The expectation change is exactly

$$\delta R_B(q, y_w) - \delta R_B(q, y_l) = \delta (R_B(q, y_w) - R_B(q, y_l)).$$

This is positive because $\delta > 0$ and the pair is B-valid. Lean proves the finite-sum identity in `expectedReward_pairwiseTransfer_eq` and then uses `linarith`. $\square$

Theorem 78 (DPO-B gradient correctness). *DPO_B_gradient_correct. Under H1 and H2, encoded as the local DPO probability-transfer hypothesis, a single DPO step on a B-valid preference pair strictly increases expected downstream entropy:*

$$\mathbb{E}_{y \sim \pi_{\theta'}}[R_B(q, y)] > \mathbb{E}_{y \sim \pi_\theta}[R_B(q, y)].$$

Proof. The local DPO hypothesis supplies a $\delta > 0$ transfer from the B-loser to the B-winner. The preceding theorem converts this transfer and B-validity directly into a strict increase in expected R_B . This is the full local training correspondence proved in Lean; the finite logit theorem above is the two-logit special case. $\square$

Theorem 79 (B-valid DPO pair moves in the correct direction). *DPO_B_correct_direction. Assume Z is finite and Y has decidable equality. Let $y_w, y_l : Y$ be feasible and satisfy*

$$H_q(y_l) < H_q(y_w).$$

For any logits $\ell : Y \rightarrow \mathbb{R}$ and step size $\eta > 0$, define ℓ' as above. Then

$$\text{LogOdds}_\ell(y_w, y_l) < \text{LogOdds}_{\ell'}(y_w, y_l).$$

Proof. The entropy dominance hypothesis $H_q(y_l) < H_q(y_w)$ implies $y_w \neq y_l$; otherwise it would reduce to $H_q(y_w) < H_q(y_w)$, impossible by irreflexivity of $<$. The remainder is exactly `dpo_logit_direction` applied to the same finite update. Lean derives the disequality from the entropy gap, simplifies the if-then-else update, and closes with `linarith`. $\square$

Training-bridge summary: B-constructed preference pairs increase winner log-odds under the finite two-logit update, and under the explicit local non-degeneracy/small-step transfer hypotheses, a DPO step strictly increases expected R_B . This justifies the DPO-style training language as a local ascent claim for the B objective.

20 RL Reward Signal Bridge

The Lean file `Creativity/Training/RLSignal.lean` connects the abstract B and D rewards to a group-relative RL signal. It defines the additive B–D diagnostic

$$R_{BD}(q, y, s) = R_B(q, y) + \lambda_D R_D(s, \mathcal{H}, u, \tau) \quad (\text{benchmarkReward}),$$

which serves the B/D rank-preservation correspondence. The *reported* Benchmark Score is the multiplicative `Rcreativity` of Section 4 (1). For a finite group of rewards r_i , the group-relative advantage is

$$A_i = \frac{r_i - \mu}{\sqrt{\frac{1}{G} \sum_j (r_j - \mu)^2 + \epsilon}}, \quad \mu = \frac{1}{G} \sum_j r_j.$$

Theorem 80 (B–D diagnostic nonnegativity). *benchmarkReward_nonneg.* If $0 \leq R_B$, $0 \leq R_D$, and $0 \leq \lambda_D$, then $0 \leq R_{BD}(q, y, s)$.

Proof. Sum of a nonnegative term and a product of nonnegatives (`add_nonneg`, `mul_nonneg`). $\square$

Theorem 81 (Positive RL denominator). *groupDenom_pos.* If $\epsilon > 0$, then

$$\sqrt{\frac{1}{G} \sum_j (r_j - \mu)^2 + \epsilon} > 0.$$

Proof. The square-root term is always nonnegative. Adding the strictly positive ϵ makes the denominator strictly positive. Lean proves this using `Real.sqrt_nonneg` and `add_pos_of_nonneg_of_pos`. $\square$

Theorem 82 (Advantage sign correctness). *advantage_sign_correct.* If $r_j < \mu < r_i$ and $\epsilon > 0$, then

$$A_j < 0 \quad \text{and} \quad 0 < A_i.$$

Proof. By `groupDenom_pos`, the denominator of each advantage is strictly positive. The numerator $r_j - \mu$ is negative, so $A_j < 0$. The numerator $r_i - \mu$ is positive, so $A_i > 0$. Lean uses `div_neg_of_neg_of_pos` and `div_pos`. $\square$

Theorem 83 (Winner advantage positivity). *B_winner_advantage_pos.* If an index i_w is a strict group winner and its reward is above the group mean, and if $\epsilon > 0$, then

$$0 < A_{i_w}.$$

Proof. The theorem uses the same denominator argument. Since the winner reward is assumed above the mean, its advantage numerator is positive. Since the denominator is positive, the quotient is positive. Lean records the strict winner hypothesis for the intended B-winner interpretation, while the algebraic sign proof uses the explicit above-mean assumption. $\square$

21 DPO Directionality Theorems

All three components of R_{train} carry a certified DPO ascent theorem under the local hypotheses **H1** (non-degeneracy: the step transfers strictly positive probability mass $\delta > 0$ from loser to winner) and **H2** (small step: the transfer is pairwise, other outputs fixed to first order):

1. `DPO_B_gradient_correct` (existing; proved above): a DPO step on a B-valid pair strictly increases $\mathbb{E}_{y \sim \pi_\theta}[R_B(q, y)]$.
2. `DPO_C_gradient_correct` (new): a DPO step on a C-valid pair strictly increases $\mathbb{E}_{y \sim \pi_\theta}[R_C(q, y)]$.
3. `DPO_D_tilde_gradient_correct` (new): a DPO step on a $\tilde{D}$ -valid pair strictly increases $\mathbb{E}_{y \sim \pi_\theta}[\tilde{R}_D(q, y)]$.

Theorem 84 (DPO-C gradient correctness). *`DPO_C_gradient_correct`. Under H1 and H2, a DPO step on a C-valid preference pair strictly increases $\mathbb{E}_{y \sim \pi_\theta}[R_C(q, y)]$.*

Proof. The proof template is reused from `DPO_B_gradient_correct`: (1) a C-valid pair gives $R_C(s_w) > R_C(s_l)$ via the utility gate (`C1_utility_gate` dual) and the novelty ordering (`cValidPair_reward_ordered`); (2) `dpo_logit_direction` applies unchanged; (3) `expectedReward_pairwiseTransfer_eq` applies unchanged, giving the expectation change $\delta(R_C(s_w) - R_C(s_l)) > 0$; (4) conclude the strict increase in $\mathbb{E}[R_C]$. $\square$

Theorem 85 (DPO- $\tilde{D}$ gradient correctness). *`DPO_D_tilde_gradient_correct`. Under H1 and H2, a DPO step on a $\tilde{D}$ -valid preference pair strictly increases $\mathbb{E}_{y \sim \pi_\theta}[\tilde{R}_D(q, y)]$.*

Proof. Identical template to the previous theorem, substituting `D_tilde_dominance` for `B1_dominance`. The generic pieces `dpo_logit_direction` and `expectedReward_pairwiseTransfer_eq` are reused unchanged. $\square$

22 Dense Trajectory Reward and 2-GRPO Quantization

22.1 Step-CUE Dense Reward

The dense trajectory reward $r_t = C(t+1) - C(t)$ is the incremental Step-CUE contribution of each reasoning step (`stepReward`). Under the per-step utility gate, PROOF-04 (`CUE_trajectory_monotone_nondecreasing`, Part III) certifies the cumulative signal never punishes a gated step: the running curve is monotone non-decreasing, so r_t is a well-posed nonnegative dense reward.

22.2 DC Bonus Necessity Theorem

Theorem 86 (PROOF-15: the terminal DC bonus is necessary). *`dense_reward_DC_bonus_incentivizes_exploration`. Consider a monotone-convergent trajectory and a diverge-converge trajectory with equal cumulative Step-CUE c . Without the bonus their total rewards tie ($\text{totalReward}(c, 0) = \text{totalReward}(c, 0)$), so the dense reward alone never prefers exploration; with any strictly positive terminal bonus b awarded to the diverge-converge trajectory,*

$$\text{totalReward}(c, 0) < \text{totalReward}(c, b).$$

Hence without r_{DC} the optimizer is indifferent to (and by tie-breaking converges on) monotone trajectories with $DC = 0$ despite maximal cumulative CUE; the bonus strictly breaks the tie toward exploration.

22.3 2-GRPO Advantage Quantization

Theorem 87 (PROOF-16: advantages quantize to $\pm 1/\sqrt{2}$). *twoGRPO_advantage_quantization_unbiased*, with *groupScale_eq* and *advantage_antisymm*. In 2-GRPO with two rollouts and rewards $r_1 > r_2$, the group scale collapses to $\sqrt{(r_1 - \mu)^2 + (r_2 - \mu)^2} = |r_1 - r_2|/\sqrt{2}$ at $\mu = (r_1 + r_2)/2$, and the group-relative advantages are exactly

$$A_1 = \frac{1}{\sqrt{2}}, \quad A_2 = -\frac{1}{\sqrt{2}},$$

independent of the reward magnitudes. The advantage is therefore a deterministic function of the signed rank alone — an unbiased signed-rank estimator, $\mathbb{E}[A_i \mid \text{rank}(y_i)] = \pm 1/\sqrt{2}$ — which is the formal core of the GRPO-is-secretly-DPO reduction used by the two-phase 2-CGRPO protocol.

23 Goodhart-Resistance Conditions

A 2-CGRPO run is valid only if all four of the following axioms hold as explicit training preconditions. Each is stated in Lean and collected in the summary conjunction below.

1. **D_tilde_no_fixed_target** — Q_{probe} is resampled every batch; no probe string is fixed across training steps.
2. **C_corpus_stationarity_condition** — the corpus $\mathcal{H}$ used to compute N_{KL} is updated on a fixed external schedule only, never by inserting policy outputs.
3. **B_utility_gate_tightness** — τ is calibrated above the entropy of incoherent baseline outputs.
4. **ProxyFaithfulness_decay_detectable** — $\rho(\tilde{R}_D, R_D)$ is monitored at every checkpoint; training halts if $\rho < \rho_{\min}$ (Part VI).

Assumption 4 (Corpus stationarity for C). *C_corpus_stationarity_condition*. The corpus $\mathcal{H}$ used to compute N_{KL} is updated on a fixed external schedule, never by inserting policy outputs: the corpus trajectory equals a schedule fixed before training, and no policy output produced during training occurs inside the corpus it is scored against (*CorpusStationarity*). Violating this axiom breaks the anti-copy and anti-padding guarantees (*C2_copy_penalty*, *C3_no_padding_exploit*) by making $\mathcal{H}$ self-referential: the policy would control the compression baseline that defines its own novelty. This is the formal Goodhart-resistance condition for R_C .

Assumption 5 (Utility gate tightness for B). *B_utility_gate_tightness*. The existing B theorems prove anti-collapse but do not block entropy-maximizing incoherence: outputs achieving high $H_q(y)$ by being vague rather than expansive. The utility threshold must satisfy

$$\tau > H_q(y_{\text{random}})$$

for every feasibility-passing but semantically incoherent output y_{random} (*IncoherentBaseline*). This is a calibration requirement: τ must be set above the entropy of incoherent baseline outputs, not just above zero, and the calibration must be validated against incoherent high-entropy baselines before training. Without it, R_B can be Goodharted by entropy-maximizing incoherence. The Lean consequences *B_incoherence_reward_below_threshold* and *B_coherent_dominates_incoherent* show that under a tight gate no incoherent baseline can reach the calibrated bar or dominate a genuinely expansive output.

Theorem 88 (Goodhart resistance summary). *Goodhart_resistance_summary.* For any admissible protocol certificates, the four Goodhart conditions hold simultaneously:

$$D_tilde_no_fixed_target \wedge C_corpus_stationarity_condition \wedge B_utility_gate_tightness \wedge ProxyFaithfulness$$

Proof. Lean assembles the conjunction from the four protocol certificates (DTildeTrainingProtocol, CTrainingProtocol, TauCalibration, ProxyMonitor), so the full protection is checkable in one place. $\square$

Validity statement. A training run that violates any of these four conditions does not have Lean-certified Goodhart resistance, and its results cannot be compared against the benchmark metrics of Part III.

Part VI

Proxy Validation and Resolution Conditions

24 Proxy Faithfulness for D

Training ascends $\tilde{R}_D$ on resampled probe prompts; the benchmark measures R_D on the frozen probe set $\mathcal{P}$. The bridge between them is conditional on an empirical assumption.

Assumption 6 (Proxy faithfulness). *ProxyFaithfulness.* After monotone calibration, the KL shift on resampled q_{probe} and R_D on the frozen $\mathcal{P}$ are monotonically related up to error bound δ : uniformly over outputs,

$$|\tilde{R}_D(s) - R_D(s, \mathcal{H}, \mathcal{P})| \leq \epsilon.$$

Unlike the purely algebraic *BEstimator/CEstimator* bounds, this is an empirical assumption that the benchmark correlation study must estimate before training begins.

Theorem 89 (Proxy rank agreement). *D_tilde_proxy_rank_agreement.* Under *ProxyFaithfulness*, if

$$\tilde{R}_D(s_1) - \tilde{R}_D(s_2) > 2\epsilon,$$

then

$$R_D(s_1, \mathcal{H}, \mathcal{P}) > R_D(s_2, \mathcal{H}, \mathcal{P}).$$

Proof. Structure mirrors *B_rank_preserved* and *C_rank_preserved*: extract the one-sided bounds from the uniform error bound with `abs_le.mp`, then chain the inequalities with `linarith`. $\square$

Theorem 90 (Proxy faithfulness decay is detectable). *ProxyFaithfulness_decay_detectable.* Let a *ProxyMonitor* record the empirical correlation $\rho(\tilde{R}_D, R_D)$ at every training checkpoint, with threshold $\rho_{\min}$ and the calibration link that faithfulness implies $\rho \geq \rho_{\min}$. If at checkpoint t the measured correlation falls below the threshold, $\rho(t) < \rho_{\min}$, then proxy faithfulness fails at t : the rank-agreement guarantee of *D_tilde_proxy_rank_agreement* no longer holds, and training must halt or the proxy must be recalibrated.

Proof. If faithfulness held at t , the calibration link would force $\rho_{\min} \leq \rho(t)$, contradicting the decay hypothesis. Lean closes with `not_le`. $\square$

Calibration requirement. $\rho_{\min}$ is a parameter of the monitor: the benchmark correlation study must estimate it *before training begins*. This theorem is the **formal Goodhart early-warning condition**: it converts the proxy faithfulness assumption into a monitorable training health invariant rather than a one-time post-hoc validation.

25 Training-to-Benchmark Correspondence

Training metric	Benchmark metric	Correspondence theorem	Resolution condition
R_B	$\hat{H}_q(y)$ on held-out tasks	B_rank_preserved	Gap $> 2\epsilon$
R_C via N_{KL} (Composability Score only)	$\hat{N}(s \mathcal{H})$ on held-out corpus	C_rank_preserved	Gap $> 2\epsilon$; not part of Benchmark Score
$\tilde{R}_D$ on resampled q_{probe}	R_D on frozen $\mathcal{P}$	D_tilde_proxy_rank_agreement	Gap $> 2\epsilon$ and $\rho \geq \rho_{\min}$
None (by design)	G_k interaction gain	Detection when gap $> 2\epsilon$ (MAS_benchmark_detects_irreducibility Agent_benchmark_detects)	Pure benchmark — correspondent

Table 1: One-to-one pairing of training metrics with benchmark metrics, the Lean correspondence theorem, and the resolution condition under which the correspondence is valid.

Remark 3 (The one asymmetry). The $\rho_{\min}$ condition in the $\tilde{R}_D$ row is the **only asymmetry** among the training–benchmark pairs. B has purely algebraic rank preservation; D’s correspondence additionally requires empirical proxy faithfulness. This asymmetry is the formal price of making R_D benchmark-only. The reported Benchmark Score is (1), not an additive $B+D$ sum; training uses $R_{\text{train}} = R_B + \tilde{\lambda}_D \tilde{R}_D$.

Part VII

The 2-CGRPO Algorithm

26 Two-Phase Protocol

Training phase

- Compute $R_{\text{train}} = R_B + \tilde{\lambda}_D \tilde{R}_D$ per output, with a fresh q_{probe} sample per batch (no R_C channel).
- Normalize to group-relative advantages via `groupDenom_pos`, `advantage_sign_correct`, and `trainReward_advantage_sign_correct`.
- Update the policy via DPO/GRPO; the directionality certificates are the B and $\tilde{D}$ theorems of Section 21.
- At every checkpoint: compute $\rho(\tilde{R}_D, R_D)$ on a small held-aside validation set; if $\rho < \rho_{\min}$, halt and flag a `ProxyFaithfulness` violation (`ProxyFaithfulness_decay_detectable`).

Evaluation phase

- Freeze the policy.
- Compute the Benchmark Score (1) (`Rcreativity`) on held-out tasks with the frozen probe set $\mathcal{P}$ (Section 4).

- Compute G_k across agent configurations from ContribBench.
- Check all resolution conditions of Section 25 (gap $> 2\epsilon$ for each metric).
- Report the ProxyFaithfulness empirical correlation as a first-class result — not an appendix detail.

Naming. **2-CGRPO** denotes: two-phase (train, then certify), Creative objective, Group Relative Policy Optimization. The “2” also signals the two-layer protection: algebraic Lean guarantees at training time, empirical proxy validation at benchmark time.

27 Validation Ledger

CUE-Layer and Structural Additions

A Benchmark Hold-Out Probe Protocol

Component D is undefined until the probe set $\mathcal{P}$ is fixed. The Component D section defines the hold-out protocol; experiments report the concrete instantiation here: sampling seed, split index, $|\mathcal{P}|$, and per-stratum counts. Under this protocol, $\mathcal{P}$ is reproducible and domain-representative by construction. Lean proves properties of $D(s, \mathcal{H}, \mathcal{P})$ for any fixed finite $\mathcal{P}$; the benchmark protocol fixes which finite $\mathcal{P}$ is used.

B Computational Approximation: Surface KL Novelty

The file `Creativity/C/SurfaceNovelty.lean` defines surface distributional novelty on finite PMFs. For output distribution p_s and corpus distribution $p_{\mathcal{H}}$,

$$N_{\text{KL}}(s \mid \mathcal{H}) := D_{\text{KL}}(p_s \parallel p_{\mathcal{H}}),$$

formalized as `surfaceNoveltyKL`. This is an empirical approximation to C, not an independent creativity dimension.

Theorem 91 (N1: utility gate). *`N1_utility_gate`. If $u < \tau$, then $R_N(q, s) = 0$.*

Proof. Identical in structure to C1: infeasibility forces the zero branch of the gated reward definition. Lean simplifies `rewardN` with `not_le.mpr`. $\square$

Theorem 92 (N2: KL nonnegativity). *`N2_kl_nonneg`. For all finite-alphabet PMFs $p_s, p_{\mathcal{H}}$,*

$$0 \leq D_{\text{KL}}(p_s \parallel p_{\mathcal{H}}).$$

Proof. This is Gibbs’ inequality for finite distributions. Lean invokes the axiom `klDivergence_nonneg` from `Creativity/Probability/KLDivergence.lean`. $\square$

Theorem 93 (N3: KL as a component of algorithmic novelty). *`N3_kl_component_bound`. If*

$$D_{\text{KL}}(p_s \parallel p_{\mathcal{H}}) + H(p_s) \leq N(s \mid \mathcal{H}),$$

then

$$D_{\text{KL}}(p_s \parallel p_{\mathcal{H}}) \leq N(s \mid \mathcal{H}).$$

Proof. Both D_{KL} and $H(p_s)$ are nonnegative by N2 and `shannonEntropy_nonneg`. Subtracting the entropy term from the assumed bound yields the claim. Lean closes with `linarith`. The cross-entropy decomposition `kl_cross_entropy_decomposition` axiomatizes the precise link $N_{\text{KL}} + H(p_s) \approx N(s \mid \mathcal{H})$ when strings are mapped to PMFs. $\square$

C Composability Scaffolding

The main text uses the Pareto-frontier theorem. The following finite constructions are Lean sanity checks showing that the objectives can prefer different outputs.

Theorem 94 (B-maximizer is not jointly dominated). *`Bmax_not_jointly_dominated`. If y_{max} satisfies*

$$B(y) \leq B(y_{\text{max}}) \quad \text{for all } y,$$

then there is no y' such that

$$B(y_{\text{max}}) < B(y') \quad \text{and} \quad C(y_{\text{max}}) \leq C(y').$$

Proof. Any joint dominator would have to be strictly better in B: $B(y_{\text{max}}) < B(y')$. But the maximality assumption gives $B(y') \leq B(y_{\text{max}})$, contradicting strict inequality. Lean uses `not_lt.mpr`. $\square$

Theorem 95 (B–C tension example). *`BC_tension_example`. There exist rewards $B, C : \text{Fin}(2) \rightarrow \mathbb{R}$ such that output 0 maximizes B, output 1 maximizes C, and the preferences are genuinely opposed:*

$$B(1) < B(0), \quad C(0) < C(1).$$

Proof. Take $B(0) = 1$, $B(1) = 0$, $C(0) = 0$, and $C(1) = 1$. Then 0 is the B-maximizer, 1 is the C-maximizer, and the two strict inequalities hold immediately. Lean proves the finite cases by case analysis on $\text{Fin}(2)$ and numeric simplification. $\square$

Theorem 96 (B–C–D tension example). *`BCD_tension_example`. There exist rewards $B, C, D : \text{Fin}(3) \rightarrow \mathbb{R}$ such that output 0 maximizes B, output 1 maximizes C, output 2 maximizes D, and the three preferences are genuinely opposed across dimensions.*

Proof. Lean constructs explicit rewards on $\text{Fin}(3)$ and verifies by finite case analysis that each component is maximized at a different index while the opposing strict inequalities hold. $\square$

Theorem 97 (Nonempty three-dimensional Pareto frontier). *`BCD_paretoFrontier_nonempty`. For any finite nonempty output type Y and rewards $B, C, D : Y \rightarrow \mathbb{R}$, there exists $y : Y$ not jointly dominated in all three components.*

Proof. Same finite-maximizer argument as the two-component case, applied to the B coordinate while ignoring the C and D inequalities in the domination witness. Lean uses `Finset.exists_mem_eq_sup'` and `not_lt.mpr`. $\square$

D Lean Appendix: B3 Scaffolding

The main text presents `B3_mixture_lb` as the general mixture anti-collapse result. Lean proves it by reduction to the algebraic lemmas below. These are scaffolding, not independent contributions.

Theorem 98 (Entropy concavity on downstream mixtures (promoted, PROOF-09)). *entropy_mixture_concavity*. For finite Y, Z , model m , prompt q , and mixture $\pi : \text{PMF}(Y)$, let $\mu_q(\pi)$ denote the downstream mixture kernel obtained by sampling $y \sim \pi$ and then $z \sim T(\cdot \mid q, y)$. Then

$$\sum_y \pi(y) H_q(y) \leq H(\mu_q(\pi)).$$

This is Shannon concavity for finite mixtures. Formerly the axiom *entropy_mixture_concavity_axiom*; now a derived theorem via concave Jensen on *Real.negMulLog* (see the proof in the B mixture section).

Theorem 99 (B3 scaffolding: weighted mixture lower bound). *B3_weighted_high_lb*. Assume Y and Z are finite. Let $\pi : \text{PMF}(Y)$ and $y_{\text{low}}, y_{\text{high}} : Y$. If

$$R_B(q, y_{\text{low}}) \leq \pi(y_{\text{high}}) R_B(q, y_{\text{high}}),$$

then

$$R_B(q, y_{\text{low}}) \leq \sum_{y:Y} \pi(y) R_B(q, y).$$

Proof. Every summand in the expectation is nonnegative, so the full finite sum is at least $\pi(y_{\text{high}}) R_B(q, y_{\text{high}})$. Composing with the hypothesis gives the claim. Lean uses `Finset.sum_nonneg`. $\square$

Theorem 100 (B3 scaffolding: strict expected dominance). *B3_expected_exceeds_dominated*. Assume Y and Z are finite. If y_{low} and y_{high} are feasible, $H_q(y_{\text{low}}) < H_q(y_{\text{high}})$, $\pi(y_{\text{high}}) > 0$, and

$$R_B(q, y_{\text{low}}) < \pi(y_{\text{high}}) R_B(q, y_{\text{high}}),$$

then

$$R_B(q, y_{\text{low}}) < \sum_{y:Y} \pi(y) R_B(q, y).$$

Proof. As above, $\pi(y_{\text{high}}) R_B(q, y_{\text{high}}) \leq \sum_y \pi(y) R_B(q, y)$. Transitivity with the strict hypothesis gives the result. Lean uses `lt_of_lt_of_le`. $\square$

E Experimental Falsifiability Map

Each experimental prediction E1–E5 of the proposal corresponds to a formal statement derivable from the Lean theorems. A failed experiment therefore falsifies an identified assumption, not the framework diffusely.

Experiment	Formal correspondent
E1 (Judge calibration)	<code>B_rank_preserved</code> , <code>C_rank_preserved</code> : rankings are estimator-noise-proof exactly when the measured gap exceeds 2ϵ ; judge miscalibration manifests as ϵ inflation.
E2 (Calibration failure replication)	<code>BContrast_distinguishes</code> : CUE-gated B maintains rank where utility-only reward models are provably indistinguishable.
E3 (Repetition vs. score)	<code>receiver_expansion_clarification_separation</code> (PROOF-06): the $R_B^{\rightarrow A}$ label is anti-correlated with surface similarity by construction, since equal-CUE outputs carry opposite expansion labels.
E4 (Discriminant validity)	<code>BCD_tension_example</code> , <code>BCD_paretoFrontier_nonempty</code> : the Pareto frontier is non-empty with distinct maximizers, so the dimensions are provably non-redundant.
E5 (Receiver stability)	<code>DC_score_existence</code> (PROOF-05) + <code>D_tilde_proxy_rank_agreement</code> : trajectory shape and proxy rank agreement are stable under the resolution conditions.

F 2-GRPO Training Protocol Specification

This appendix makes the Lean-certified guarantees operationally executable for the planned training run. The formal object instantiated is `DTildeTrainingProtocol` together with the calibration structures of Parts IV and V.

1. **$\rho_{\min}$ calibration.** Before training begins, run the benchmark correlation study: sample checkpoint-free model outputs, compute $(\tilde{R}_D, R_D)$ pairs on a held-aside validation set, and set $\rho_{\min}$ from the empirical correlation distribution (e.g. its lower confidence bound). This discharges the precondition of `ProxyFaithfulness_decay_detectable`.
2. **τ calibration.** Generate incoherent high-entropy baselines (feasibility-passing shuffled/noise outputs), estimate $H_q(y_{\text{random}})$, and set τ strictly above it. This discharges `B_utility_gate_tightness` and blocks exploitation path (iii) of the Structural Goodhart Theorem.
3. **Corpus stationarity schedule.** Fix the external corpus update schedule before training; policy outputs are never inserted into $\mathcal{H}$. This discharges `C_corpus_stationarity_condition`.
4. **Probe resampling.** Resample Q_{probe} fresh every batch; no probe string persists across steps (`D_tilde_no_fixed_target`).
5. **Rollout and advantage computation.** Two rollouts per prompt; advantages quantize to $\pm 1/\sqrt{2}$ (`twoGRPO_advantage_quantization_unbiased`), so the update is a signed-rank (DPO-equivalent) step at roughly 1/8 of the 16-rollout cost.
6. **Checkpoint monitoring.** At every checkpoint compute $\rho(\tilde{R}_D, R_D)$ on the held-aside validation set; if $\rho < \rho_{\min}$, halt and flag a `ProxyFaithfulness` violation. Report the final correlation as a first-class result.

G Known Gaps Registry

Central, auditable list of every statement the Lean development assumes rather than proves, with its promotion path. All other results cited in this document are machine-checked theorems with no sorry.

1. **KG-1 (analytic imports).** `klDivergence_nonneg` (Gibbs inequality) and `kl_cross_entropy_decomposition` (cross-entropy link) in `Creativity/Probability/KLDivergence.lean`. Promotion path: derive from Jensen on the convex function $x \mapsto x \log x$ over the support, mirroring the promotion of `entropy_mixture_concavity` (PROOF-09). Note that `maxEntropy_finite_alphabet` is already *derived from* KG-1 rather than assumed separately.
2. **KG-2 (statistical import).** `plugIn_bias_axiom` (Miller–Madow bias bound, PROOF-12). Promotion path: Hoeffding’s inequality plus Lipschitz continuity of $-x \log x$; blocked on a Lean 4 concentration-of-measure dependency.
3. **KG-3 (protocol certificates).** `D_tilde_no_fixed_target` (probe resampling) and `B_utility_gate_tightness` (τ calibration) are certificates carried by protocol structures: they are discharged by *running the protocol correctly* (Appendix F), not by mathematics. Their violation consequences are machine-checked by the Structural Goodhart Theorem (PROOF-13).
4. **KG-4 (resolved).** Formerly registered gaps now closed: `entropy_mixture_concavity_axiom` (promoted, PROOF-09), `productKernel_joint_entropy_eq_sum` (promoted, PROOF-10), and `gain_zero_iff_productKernel` (removed as incorrect and replaced by `gain_zero_iff_joint_eq_max` plus `productKernel_additivity_gain_pos`, PROOF-11).
5. **KG-5 (optional shifted threshold).** f for the calibration variant `CUE_shifted` / `Rcreativity_shifted` — not part of the canonical definition (1). Empirically calibrated per domain; all shifted theorems hold for every real f .
6. **KG-6 (Benchmark Score hyperparameters).** α (expansion-bonus weight; also absorbs bits/nats residual scale), δ_D (D-gate threshold), and λ_G (interaction-gain weight) are empirically calibrated with no canonical closed form. Status: registered assumptions, not proved.

Layer	Lean theorem	Main assumptions
Entropy	shannonEntropy_nonneg	finite support PMF
Entropy	shannonEntropy_point_mass	finite support PMF, point mass
Entropy	shannonEntropy_lt_iff	definitional unfolding
B helper	rewardB_of_feasible	feasible output
B helper	rewardB_of_infeasible	infeasible output
B helper	rewardB_nonneg	entropy nonnegativity and gate cases
B helper	mem_feasibleSet_iff	finite filtered feasible set
B objective	B1_dominance	feasible pair, entropy gap
B objective	B2_no_collapse	finite Y, Z , feasible nonempty set, entropy gap
B mixture	B3_mixture_lb	mass condition $R_B(y_{\text{low}}) \leq \pi(y_{\text{high}})R_B(y_{\text{high}})$
B mixture	entropy_mixture_concavity	theorem (PROOF-09, promoted): concave Jensen on <code>negMulLog</code>
B mixture	B3b_jensen_anti_collapse	now axiom-free via <code>entropy_mixture_concavity</code>
C objective	C1_utility_gate	$u < \tau$ (Composability Score only)
C objective	C2_copy_penalty	copy-minimal compressor (Composability Score only)
C objective	C3_no_padding_exploit	normalized compressor padding bound (Composability Score only)
C objective	C3_no_trivial_padding_gain	zero-padding cost, normalized compressor (Composability Score only)
N surface	N1_utility_gate	$u < \tau$
N surface	N2_kl_nonneg	Gibbs inequality (finite PMF axiom)
N surface	N3_kl_component_bound	KL plus entropy bounded by novelty
N surface	klDivergence_nonneg	finite PMF KL nonnegativity axiom
N surface	kl_cross_entropy_decomposition	cross-entropy link to C
D probe-relative	D1_utility_gate	$u < \tau$, fixed finite $\mathcal{P}$
D probe-relative	D2_dominance	feasibility, strict deformation gap
D probe-relative	D3_rewardC_bounded	copy-minimal compressor (Composability Score only)
D probe-relative	D3_exploration_transformation_bound	ProbeCompressor (Composability Score only)
D probe-relative	D3_rewardD_bound_standalone	ProbeCompressor ; Benchmark Score, R_C -free
D probe-relative	D4_zero_for_corpus_copy	append-self invariance
D probe-relative	D4_zero_reward_for_corpus_copy	feasibility plus D4
B baseline	BBase_indistinguishable	feasible pair, equal utility
B baseline	BContrast_distinguishes	feasible pair, equal utility, entropy gap
C baseline	CBase_infeasible_can_maximize	$u < \tau$, positive novelty (Composability Score only)
B estimator	B_rank_preserved	uniform error, true gap $> 2\epsilon$
C estimator	C_rank_preserved	uniform error, true gap $> 2\epsilon$
DPO bridge	dpo_logit_direction	distinct winner/loser, $\eta > 0$
DPO bridge	DPO_B_correct_direction	feasible B pair, entropy gap, $\eta > 0$
DPO bridge	dpo_gradient_increases_logodds	$\beta > 0$, positive sigmoid term, H1, $\eta > 0$
DPO bridge	softmax_monotone_in_logit_diff	two-logit softmax, $\delta > 0$
DPO bridge	dpo_step_increases_expected_RB	B-valid pair, positive probability transfer
DPO bridge	DPO_B_gradient_correct	local DPO transfer hypothesis (H1/H2)
RL signal	benchmarkReward_nonneg	nonnegative B/D rewards and λ_D (diagnostic)
RL signal	trainReward_nonneg	nonnegative B/ $\bar{D}$ training signal (no R_C)
RL signal	trainReward_neq_benchmarkReward	train/benchmark split machine-checked
RL signal	groupDenom_pos	positive advantage stabilizer ϵ
RL signal	advantage_sign_correct	rewards below/above group mean
RL signal	B_winner_advantage_pos	winner above group mean, $\epsilon > 0$

Table 2: Lean-checked claims and their principal assumptions (objective, estimator, and DPO layers).

Layer	Lean theorem	Main assumptions
MAS	MAS_irreducibility	positive interaction gain
MAS benchmark	MAS_benchmark_detects_irreducibility	interaction gain $> 2\epsilon$
MAS k -agent	kAgent_irreducibility	positive k -agent interaction gain
MAS k -agent	kAgent_benchmark_detects	k -agent gain $> 2\epsilon$
MAS graph	CommGraph	irreflexive directed communication graph
MAS graph	nonProduct_base_case	faithfulness plus edge $0 \rightarrow 1$
MAS graph	connectedMAS_nonProductKernel	faithfulness plus at least one edge
MAS graph	gain_zero_iff_joint_eq_max	theorem (PROOF-11, corrected): zero gain iff $H_J = \max_i H_i$; old axiom removed
MAS graph	productKernel_additivity_gain_pos	additive joint entropy, positive marginals $\Rightarrow$ positive gain
MAS graph	gain_biconditional	nonnegative gain plus corrected degeneracy characterization
O-info	oInformation3_negative_of_pair_sum_lt	pair-sum below triple term
O-info	oInformation3_positive_of_triple_lt_pair_sum	triple term below pair-sum
O-info	equalMarginals_gain_implies_synergy	equal-marginal sign condition
O-info	negativeGain_implies_redundancy	collapse sign condition
O-info bridge	productKernel_joint_entropy_eq_sum	theorem (PROOF-10, promoted): via shannonEntropy_jointPMF
O-info bridge	productKernel_implies_redundancy	strictly positive marginal entropies
Composability	Bmax_not_jointly_dominated	B maximizer
Composability	paretoFrontier_nonempty	finite nonempty output type
Composability	BC_tension_example	explicit Fin(2) construction
Composability	BCD_tension_example	explicit Fin(3) construction
Composability	BCD_paretoFrontier_nonempty	finite nonempty output type
Composability	paretoFrontier_nonempty_BD	finite nonempty output type; Benchmark Score, R_C -free
Estimator calibration	plugIn_bias_axiom	registered import (PROOF-12, KG-2): Miller–Madow-scale bound
Estimator calibration	entropy_ranking_sample_requirement	true gap $> Y /n$
Benchmark spec	benchmark_can_detect_S2_dominance	true S2–S1 gap $> 4\epsilon$

Table 3: Lean-checked claims and their principal assumptions (multi-agent, composability, and benchmark layers).

Layer	Lean theorem	Main assumptions
D proxy	D_tilde_utility_gate	$u < \tau$
D proxy	D_tilde_nonneg	gated KL nonnegativity
D proxy	D_tilde_zero_for_identical	policy matches frozen reference on probe
D proxy	D_tilde_no_fixed_target	protocol axiom : probe batch resampled, no fixed probe string
D proxy	D_tilde_dominance	feasibility, strict proxy-reward gap
DPO bridge	DPO_C_gradient_correct	C-valid pair, local DPO transfer (H1/H2)
DPO bridge	DPO_D_tilde_gradient_correct	$\bar{D}$ -valid pair, local DPO transfer (H1/H2)
Proxy bridge	D_tilde_proxy_rank_agreement	proxy faithfulness, proxy gap $> 2\epsilon$
Proxy bridge	ProxyFaithfulness_decay_detectable	monitor calibration link, $\rho < \rho_{\min}$
RL signal	trainReward_nonneg	nonnegative B/C components and weights
RL signal	trainReward_advantage_sign_correct	train rewards below/above group mean
RL signal	trainReward_neq_fullReward	weighted D components differ
Goodhart	B_utility_gate_tightness	calibration axiom : $\tau > H_q(y_{\text{random}})$
Goodhart	C_corpus_stationarity_condition	protocol axiom : external corpus schedule only
Goodhart	Goodhart_resistance_summary	all four protocol certificates

Table 4: Lean-checked claims and their principal assumptions (training-signal, proxy, and Goodhart layers).

Layer	Lean theorem	Main assumptions	Status
Foundation	<code>maxEntropy_finite_alphabet</code>	Gibbs inequality	Machine-checked
Foundation	<code>klDivergence_uniform_eq</code>	(<code>klDivergence_nonneg</code>)	Machine-checked
Foundation	<code>sum_toReal_eq_one</code>	finite nonempty alphabet	Machine-checked
Foundation	<code>shannonEntropy_eq_sum_negMulLog</code>	finite PMF	Machine-checked
CUE	<code>cue, cue_nonneg</code>	definitional	Machine-checked
CUE	<code>CUE_voi_equivalence</code> (PROOF-01)	nonnegative Brier delta, positive bit length	Machine-checked
CUE	<code>CUE_voi_equality</code> (PROOF-01)	proper-scoring-rule bound field	Machine-checked
CUE	<code>shannon_upper_bound_normalization</code> (PROOF-02)	calibrated binary saturation	Machine-checked
CUE	<code>CUE_primary_gate</code>	$\text{VOI} \leq \text{posterior entropy field}$	Machine-checked
CUE	<code>receiver_expansion_clarification_separation</code> (PROOF-06)	gate definition (Composability Score only; superseded by shifted form)	Machine-checked
CUE shifted	<code>CUE_shifted_bound</code>	explicit Fin(2) construction	Machine-checked
CUE shifted	<code>CUE_shifted_sign_iff</code>	Shannon ceiling minus f ; Benchmark Score, R_C -free	Machine-checked
CUE shifted	<code>receiver_expansion_bounded</code>	threshold clearance; Benchmark Score, R_C -free	Machine-checked
CUE shifted	<code>Rcreativity_sign_matches_CUE_shifted</code>	normalized entropy ratio; Benchmark Score, R_C -free	Machine-checked
CUE shifted	<code>Gk_gated_by_CUE_shifted</code>	$\alpha > 0, r_b \in [0, 1]$; Benchmark Score, R_C -free	Machine-checked
Trajectory	<code>CUE_trajectory_monotone_nondecreasing</code> (PROOF-04)	bridge implication, $f < \text{CUE}$; Benchmark Score, R_C -free	Machine-checked
Trajectory	<code>DC_score_existence</code> (PROOF-05)	per-step gate $r_t \geq 0$	Machine-checked
Trajectory	<code>monotone_increasing_not_DC</code> (PROOF-05)	explicit tent construction	Machine-checked
Trajectory	<code>step_CUE_gamma_positivity_characterization</code> (PROOF-14)	strictly increasing trajectory	Machine-checked
Trajectory	<code>dense_reward_DC_bonus_incentivizes_exploration</code> (PROOF-15)	fitted exponential-plus-linear curve	Machine-checked
		typical cumulative CUE, positive bonus	Machine-checked

Table 5: CUE-layer additions (foundation, gate, and trajectory diagnostics) with verification status. “Machine-checked” means proved in Lean with no `sorry`.

Layer	Lean theorem	Main assumptions	Status
MAS bridge	<code>Gk_implies_CUE_improvement</code> (PROOF-07)	calibration link, $\text{gain} > 2\varepsilon_{\text{cal}}$	Machine-checked
MAS bridge	<code>Gk_implies_CUE_improvement_sup</code> (PROOF-07)	calibration link, $\text{gain} > 2\varepsilon_{\text{cal}}$	Machine-checked
DPS	<code>dual_process_separation_principle</code> (PROOF-03)	live-only reward construction	Machine-checked
DPS	<code>Rtrain_cannot_goodhart_frozen</code> (PROOF-03)	live-only reward construction	Machine-checked
Goodhart	<code>rank_reversal_refutes_agreement</code> (PROOF-13)	rank-reversal witness	Machine-checked
Goodhart	<code>reversal_instance_exists</code> (PROOF-13)	explicit two-candidate construction	Machine-checked
Goodhart	<code>structural_goodhart_collapse</code> (PROOF-13)	composition of the above	Machine-checked
Commensurability	<code>rewardB_le_maxEntropy</code> (PROOF-08)	max-entropy bound	Machine-checked
Commensurability	<code>lambda_normalization_equalizes_maxima</code> (PROOF-08)	positive length and $\log \Sigma $ (Composability Score only)	Machine-checked
Commensurability	<code>lambda_D_normalization_standalone</code>	probe size/length only; Benchmark Score, R_C -free	Machine-checked
2-GRPO	<code>groupScale_eq</code> (PROOF-16)	two-rollout group	Machine-checked
2-GRPO	<code>advantage_antisymm</code> (PROOF-16)	two-rollout group	Machine-checked
2-GRPO	<code>twoGRPO_advantage_quantization_unbiased</code> (PROOF-16)	$r_2 < r_1$	Machine-checked
MAS promoted	<code>jointPMF_apply</code> (PROOF-10)	finite alphabet	Machine-checked
MAS promoted	<code>shannonEntropy_jointPMF</code> (PROOF-10)	finite alphabet	Machine-checked
Known gap	<code>klDivergence_nonneg</code> (KG-1)	Gibbs inequality	Registered import
Known gap	<code>kl_cross_entropy_decomposition</code> (KG-1)	cross-entropy link	Registered import
Known gap	<code>plugIn_bias_axiom</code> (KG-2)	Miller–Madow bound	Registered import

Table 6: Structural additions (dual-process separation, Goodhart collapse, commensurability, 2-GRPO, MAS promotions) with verification status. “Registered import” means an explicitly documented trusted axiom listed in the Known Gaps Registry (Appendix G).